Convolution and Image Filtering

Credit: 1. Stephen F Elston, CSCI E-25 Computer Vision

HARVARD
Extension School

Credit: 2. S. Seitz

Image Filtering

Almost all computer vision pipelines require filtering

- Without good image data preparation even the most state of the art CV algorithms will not produce good results
- Classical filters apply the mathematical operation of **convolution**
  - Noise suppression
  - Edge and corner enhancement
  - Anti-aliasing
- We can understand the effects of filtering through **spectral analysis**
  - Filter changes power spectrum of image
  - Typical filters reduce high-frequency energy – noise
  - Higher frequency components include sharp edges and corners – typically blurred by filtering

Image Filtering

Key points for this lesson

- How convolution works in 1-d, 2-d and higher dimension
- How to apply padding, stride and tiling for convolution
- Constructing convolutional filter kernels
- The Fourier transform and frequency components of images
- The relationship between filtering and the spectrum of images

Convolution

Convolution

Convolution operations are a computationally efficient method of applying filters

- Convolutional kernel weights determine characteristics of filters
  - Filtering noise
  - Creating and enhancing features
- Convolutional filtering computationally efficient
  - High volume applications
  - Real-time CV

Convolution

Convolution operations are a computationally efficient method of applying filters

- Discrete convolutional kernels have a **span**, measured in points
- Kernel moves over image, a **stride length** at a time, producing filtered output
- Typically stride $\leq$ span
- Tiled filters do not overlap, stride = span

Convolution

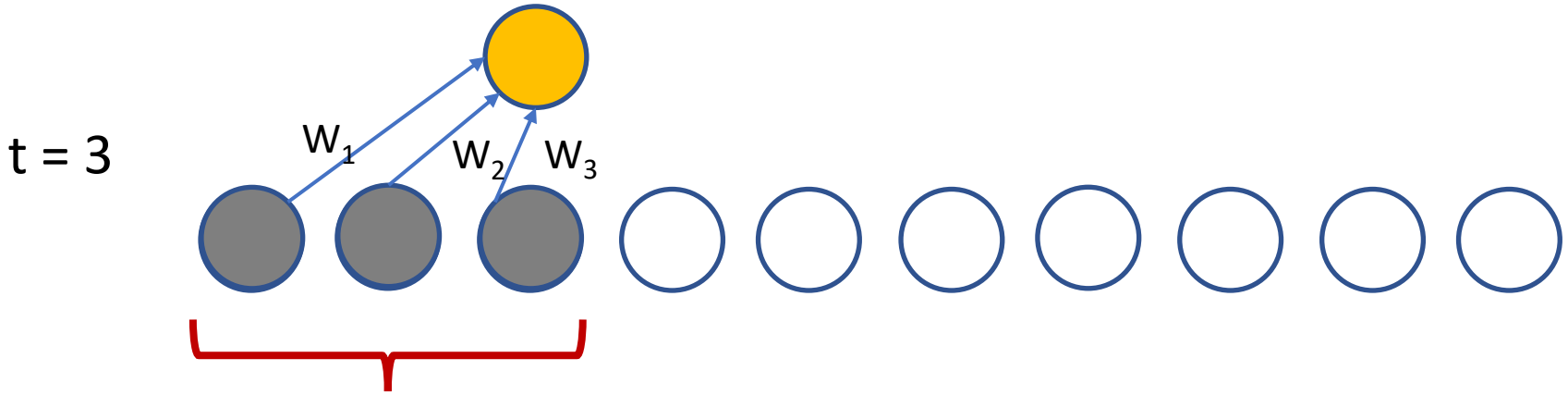
Convolution operations are a computationally efficient method of applying filters to discretely

- Convolution kernel is moved along the input tensor
  - Kernel has a small span compared to dimension of input tensor
  - Number of kernel dimensions $\leq$ number of input tensor dimensions
  - At each step a weighted output value is computed
  - Output tensor can have different number of dimensions from input tensor
- 1-D convolutions are a simple, but useful example
  - Time series data
  - Speech
  - Medical signals

1-D Convolution

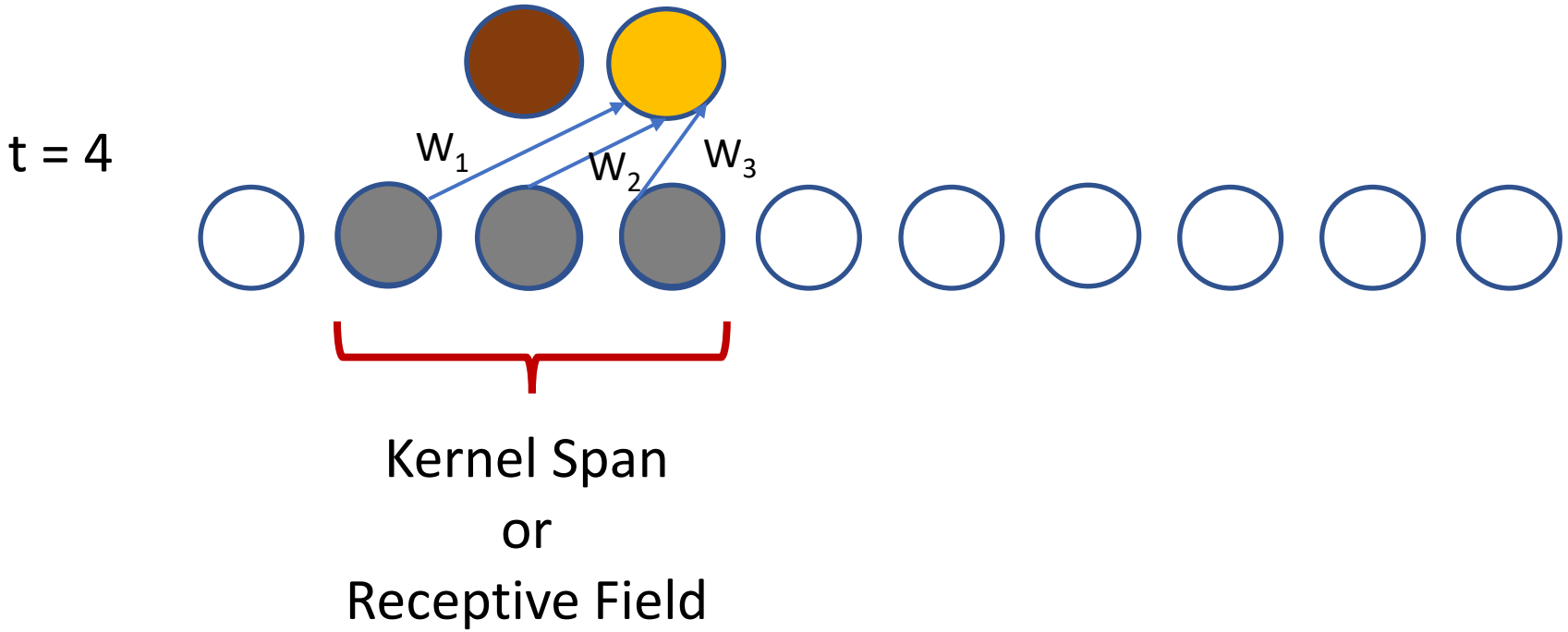
1-D CNNs are a simple, but useful example

- Consider a convolutional kernel with a **span** of 3 and **stride** 1
- Kernel defined by **weights** $[w_1, w_2, w_3]$
- The convolved value is the weighted sum over the span of the **convolutional kernel**



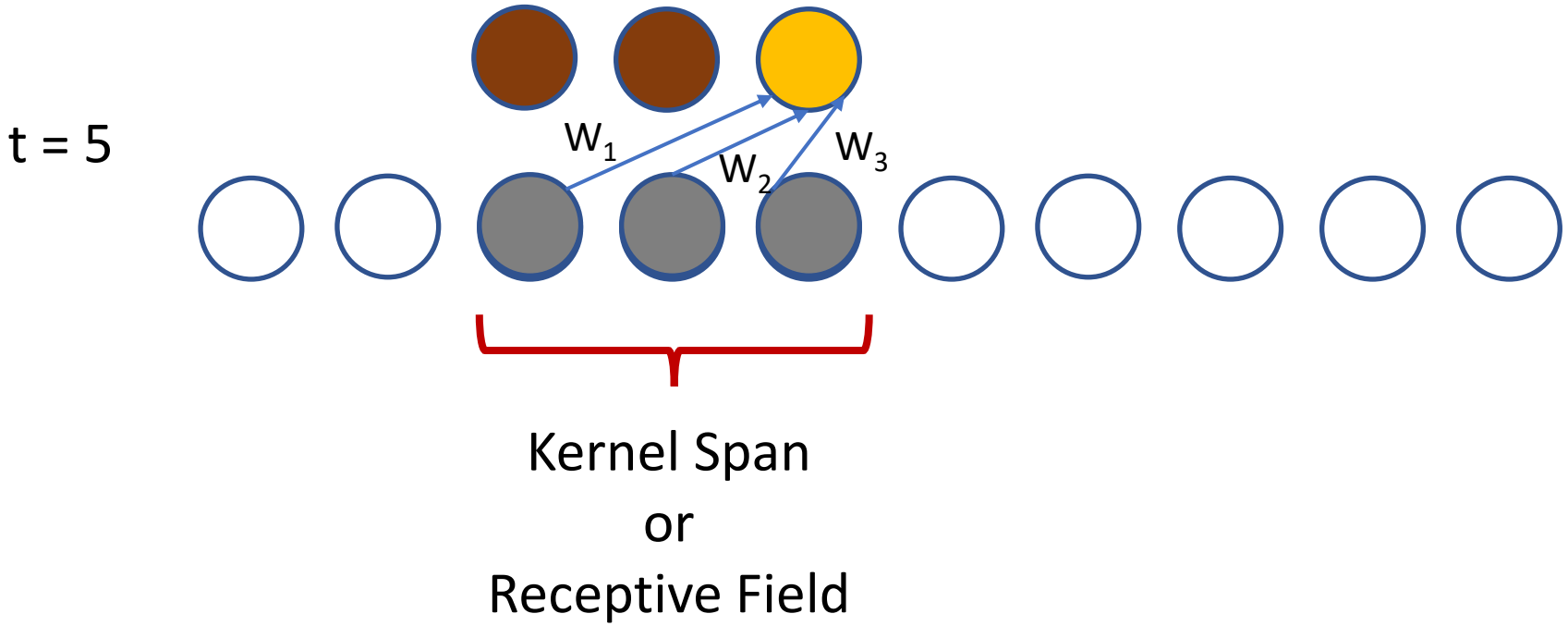
1-D Convolution

- 1-D CNNs are a simple, but useful example
- Move kernel by **stride** 1
- The convolutional kernel moves one sample per step



1-D Convolution

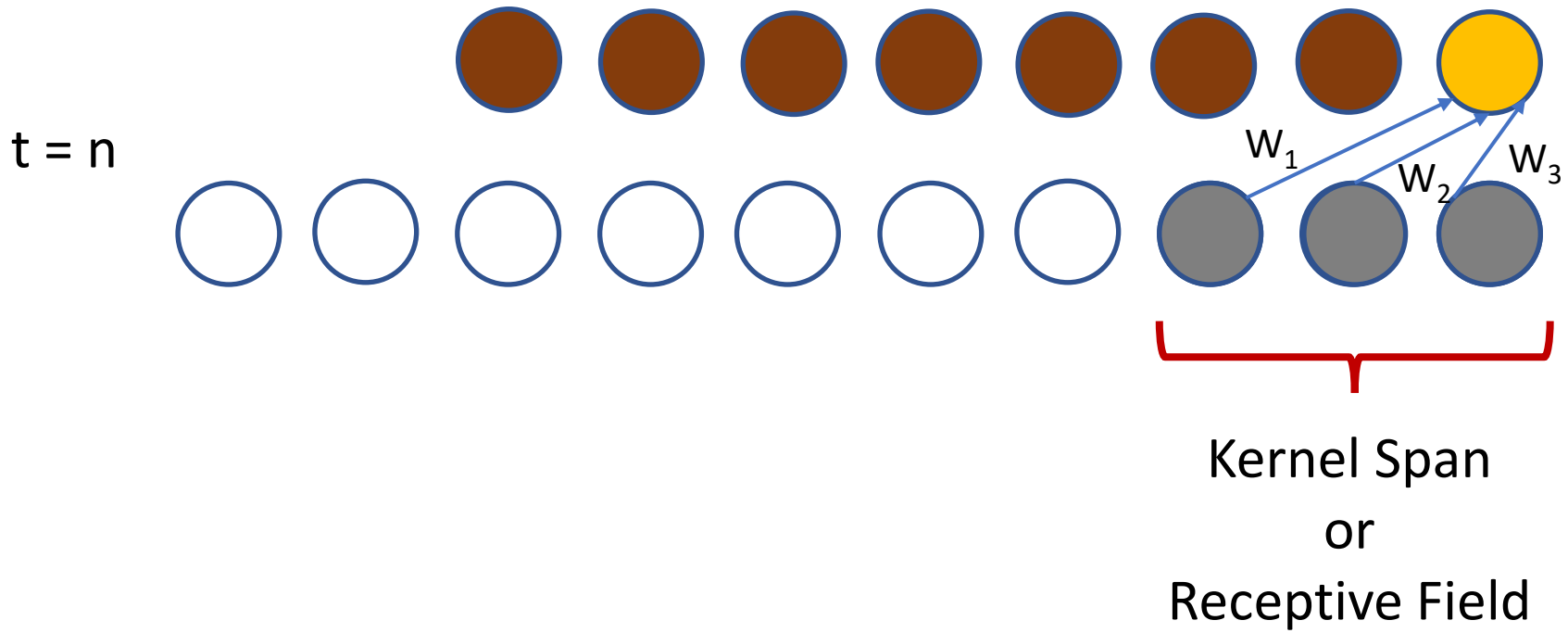
- 1-D CNNs are a simple, but useful example
- Continue to produce one output value per time step



1-D Convolution

1-D CNNs are a simple, but useful example

- Convolution ends when end of series reached
- For odd span, output is $n - (span + 1)/2$, shorter than original series



1-D Convolution

Mathematically, express 1-d convolution as a weighted sum over a set of discrete kernel values:

$$s(t) = (x * k)(t) = \sum_{\tau=t-a}^{t+a} x(\tau)k(t-\tau)$$

Where:

$s(t) =$ the output of the convolution operation at time t

$x =$ series of values

$k =$ convolution kernel

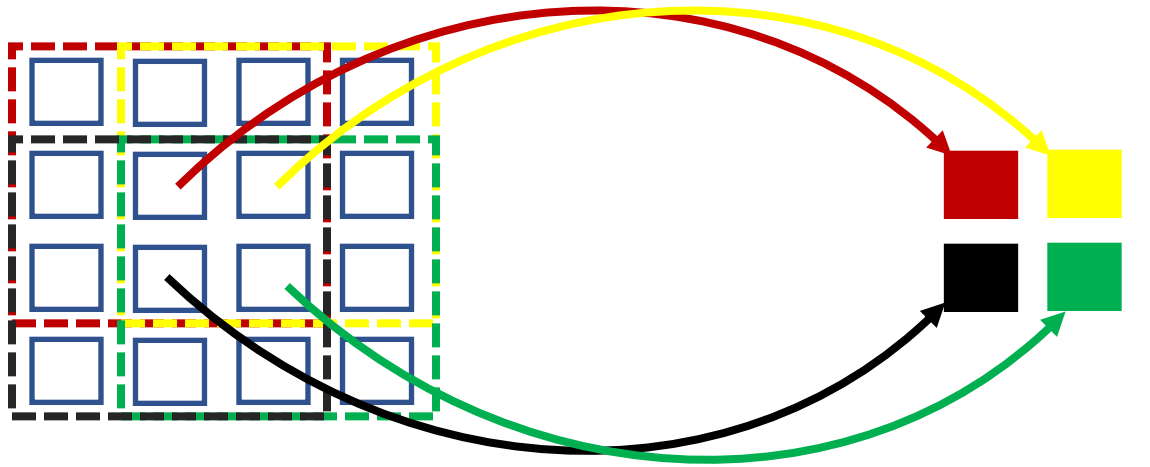
$*=$ convolution operator

$\tau =$ time difference of the convolution operator

$a = \frac{1}{2}(span\ - 1)$, for odd length operator

2-D Convolution

- 4 x 4 input tensor
- 3 x 3 convolution operator
- 2 x 2 output tensor



Input

Output

2-D Convolution

We can extend convolution to higher dimensions

- Mathematically, we express 2-d convolution as a weighted sum over a discrete rectangular kernel
- For square kernel, $K(i,j)$, of odd dimensions $a \times a$, and p=(a-1)/2, convolution can be expressed

$$S(i,j) = (I * K)(i,j) = \sum_{m=-p}^{p} \sum_{n=-p}^{p} I(i+m, j+n) K(m,n)$$

- Where S, I and K are now 2-dimensional tensors

Algebraic Properties of Convolution

Convolution operations have useful **algebraic properties**

Name	Property
Commutativity	$g * k = k * g$
Associativity	$k * (g * h) = (k * g) * h$
Distributivity	$k * (g + h) = (k * g) + (k * h)$
Associativity of scalar multiplication	$\alpha(k * g) = \alpha\, k * g$

Notice that these properties are identical to the familiar algebraic properties of multiplication

Stride, Tiling and Padding for Convolution

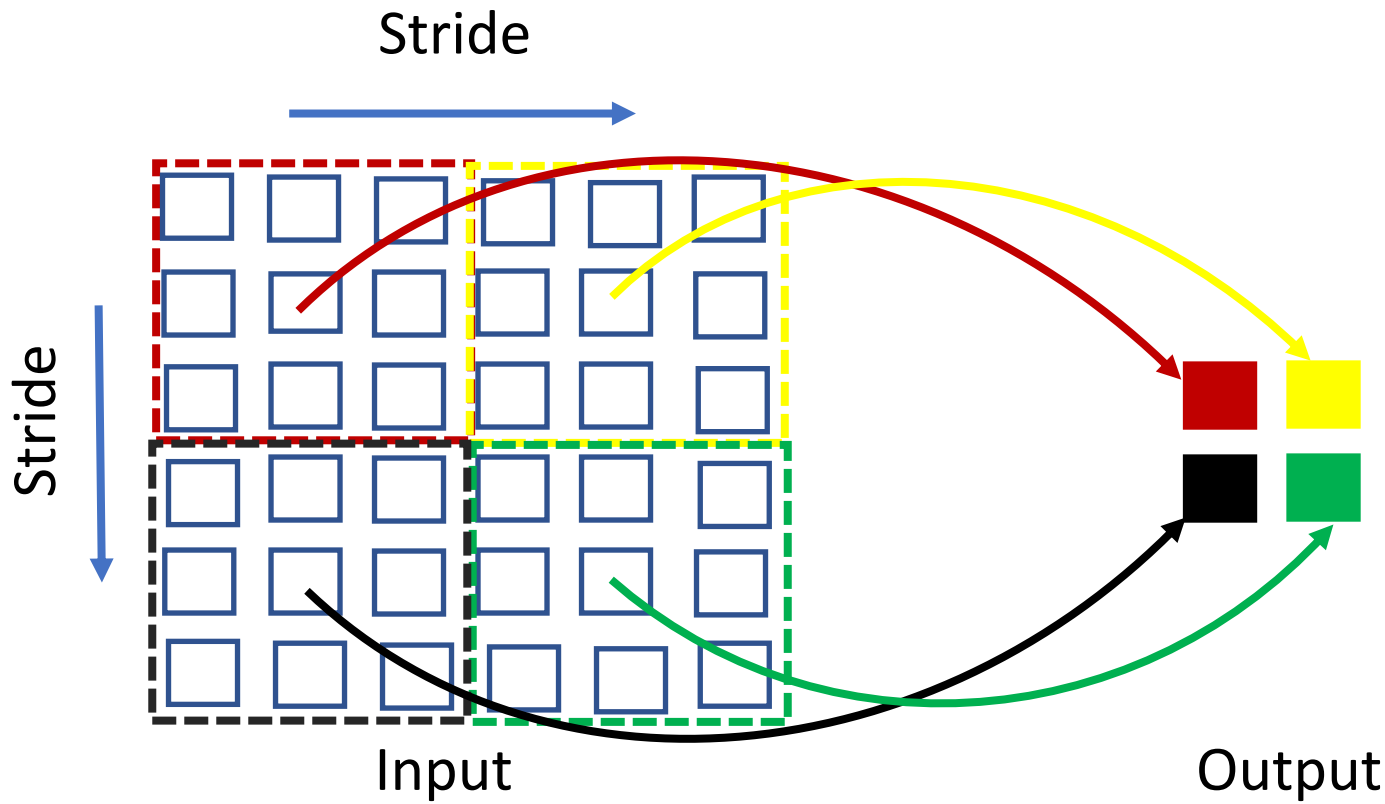
Stride and Tiling

Do we always move the convolution kernel by 1 data value?

- **No!**
  - May not need the full resolution of the input
  - Or, change the dimension of the input tensor
- Stride > 1 **down-samples** the data
  - Output of lower resolution
- Stride < 1 **up-samples** the data
  - Output of higher resolution
- **Stride can be fractional** to create the required output dimension

Stride and Tiling

Tiling is a special case when stride = span:



Stride >1 reduces dimensionality

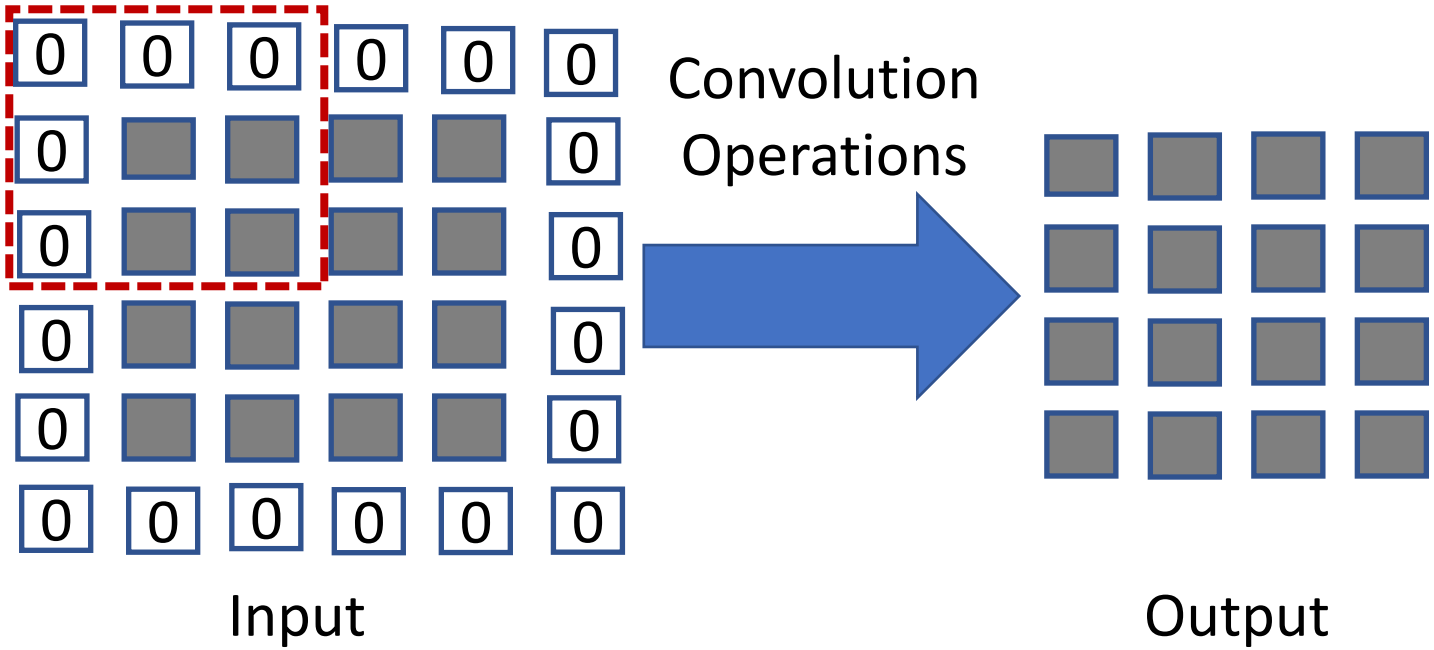
Padding for Convolution

How can we best deal with **edges of the input** tensor when performing convolution?

- A **valid convolution** confines the kernel to the input tensor dimensions
  - For odd kernel shape, output dimension is (span + 1)/2 less than input dimension
  - After many convolution layers, dimension is reduced further
- We can **zero pad** the input tensor
  - Dimensionality is maintained

Padding for Convolution

Example: 4x4 input tensor with 3x3 kernel



- Dimension of output tensor = dimension of input tensor
- 0 values have little effect with max pooling

Commonly Used Convolution Kernels

Common Convolution Kernels

A **moving average kernel** has equal weights

- Example, a 3 x 3 moving average kernel

$$K(i,j) = \frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

- This is also known as a **blur filter** or **box blur**
- Notice the **normalization** so weights add to 1.0

Common Convolution Kernels

A **Gaussian kernel** smooths an image

- Weights computed from Gaussian function with span parameter σ:

$$G(x,y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

- Example, a 3 x 3 Gaussian kernel, $\sigma^2 = 1$

$$K(i,j) = \frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

- Then Gaussian kernel limits the bandwidth and blurs the image
- The larger σ^2 the narrower the bandwidth of the Gaussian kernel

Common Convolution Kernels

The **Sobel kernel** takes the first derivative of the image

- Compute the partial derivatives of the image, G, along each axis

$$\frac{\partial G}{\partial y} = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix} * G, \qquad \frac{\partial G}{\partial x} = \begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix} * G$$

- Here, $*$, is the convolution operator
- Notice the sum of the weights is 0
- Other derivative kernels can be derived

Common Convolution Kernels

The Sobel kernel is an example of a **separable kernel**

- Notice that we can write a Sobel kernel as the product of two 1-dimensional kernels

$$\begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix} \begin{bmatrix} 1 & 2 & 1 \end{bmatrix}, \qquad \begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & -1 \end{bmatrix}$$

- For computational efficiency a 2-dimensional separable kernel can be applied as the convolution of 2 1-dimensional kernels

Median Filter

Median filter is a special case

- Median filter outputs median value of the pixel values within patch
  - Typically use odd patch sizes – e.g., 3 x 3, 5 x 5, 7 x 7
- Median filter is not easily formulated using convolution
  - Use an incremental update algorithm for computational efficiency
- Median filter has interesting properties
  - Suppresses noise
  - Sharpens edges and corners
- Median filter can alter image
  - Response is not isotropic – not same along diagonals of image
  - Can move edges and corners

Image filtering (or convolution)

- Image filtering: compute function of local neighborhood at each position

- Really important!
  - Enhance images
    - Denoise, resize, increase contrast, etc.
  - Extract information from images
    - Texture, edges, distinctive points, etc.
  - Detect patterns
    - Template matching
  - Deep Convolutional Networks

Example: box filter

$K[\cdot\,,\cdot\,]$

$$\frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Slide credit: David Lowe (UBC)

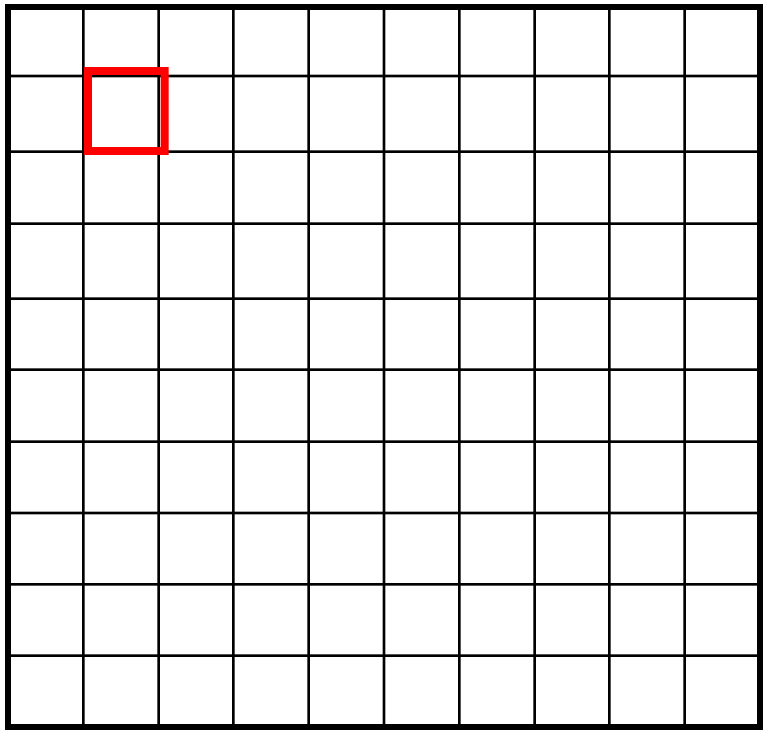
Image filtering

$K[\cdot\,,\cdot\,]$

$$\frac{1}{9}\begin{bmatrix}1&1&1\\1&1&1\\1&1&1\end{bmatrix}$$

$I[.,.]$

$S[.,.]$



$$S(i,j) = (I * K)(i,j) = \sum_{m=-p}^{p}\sum_{n=-p}^{p} I(i+m, j+n)K(m,n)$$

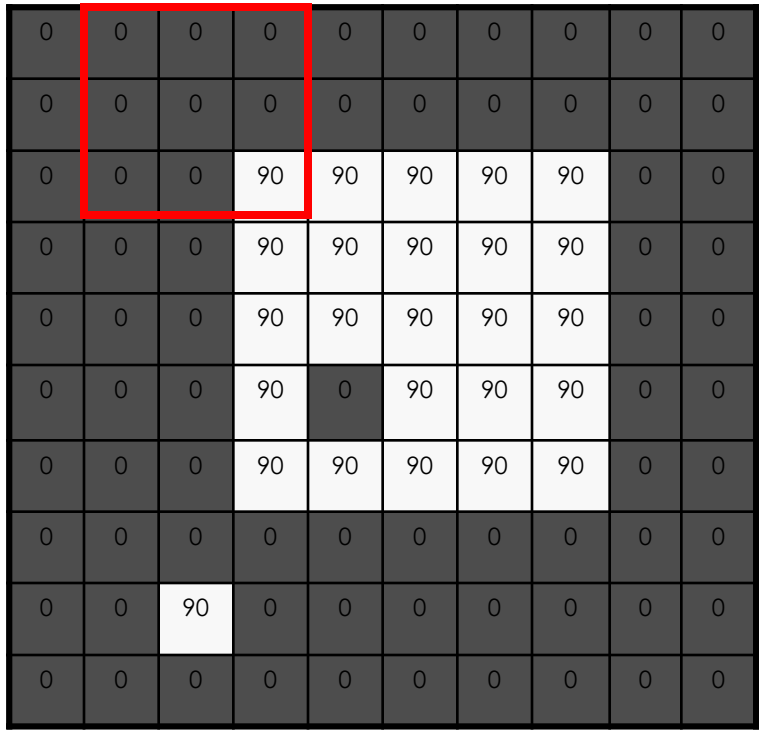
Credit: S. Seitz

Image filtering

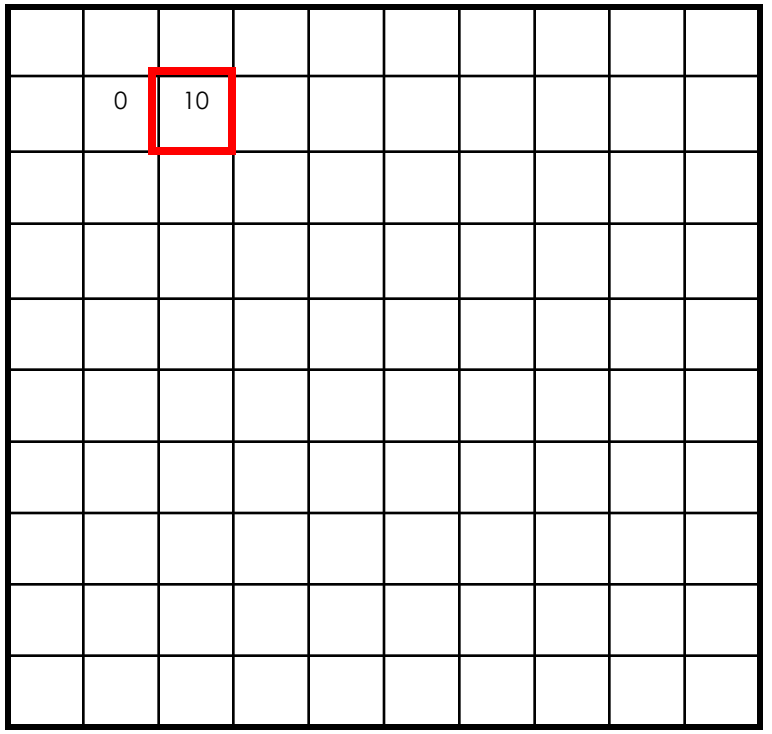
$K[\cdot\,,\cdot\,]$

$$\frac{1}{9}\begin{bmatrix}1&1&1\\1&1&1\\1&1&1\end{bmatrix}$$

$I[.,.]$



$S[.,.]$



$$S(i,j) = (I * K)(i,j) = \sum_{m=-p}^{p}\sum_{n=-p}^{p} I(i+m, j+n)K(m,n)$$

Credit: S. Seitz

Image filtering

$K[\cdot\,,\cdot\,]$ $\frac{1}{9}$

1	1	1
1	1	1
1	1	1

$I[.,.]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$S[.,.]$

	0	10	20						

$$S(i,j) = (I * K)(i,j) = \sum_{m=-p}^{p} \sum_{n=-p}^{p} I(i+m, j+n)K(m,n)$$

Credit: S. Seitz

Image filtering

$K[\cdot\,,\cdot\,]$

$$\frac{1}{9}\begin{bmatrix}1&1&1\\1&1&1\\1&1&1\end{bmatrix}$$

$I[.,.]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$S[.,.]$

	0	10	20	30					

$$S(i,j) = (I * K)(i,j) = \sum_{m=-p}^{p}\sum_{n=-p}^{p} I(i+m, j+n)K(m,n)$$

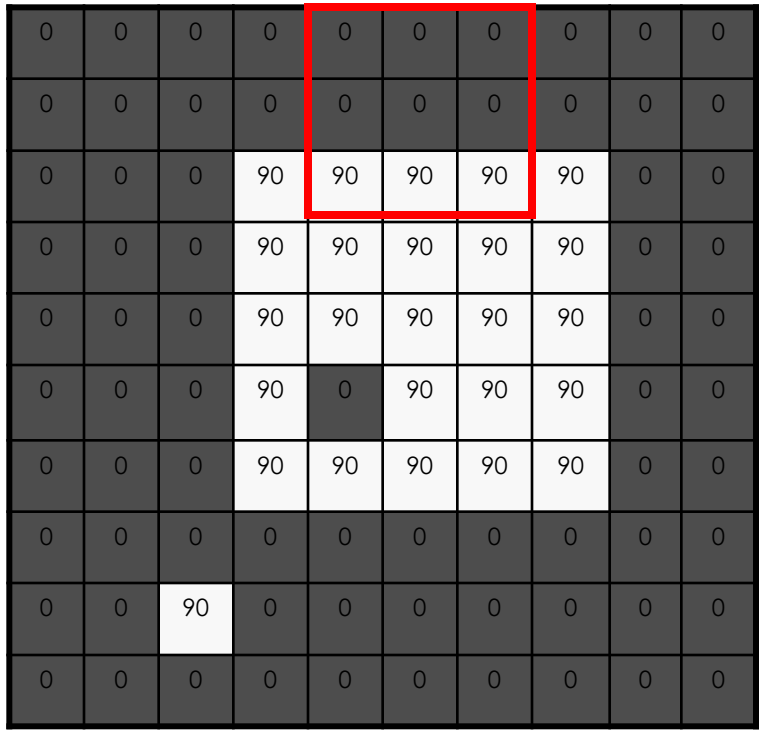
Credit: S. Seitz

Image filtering

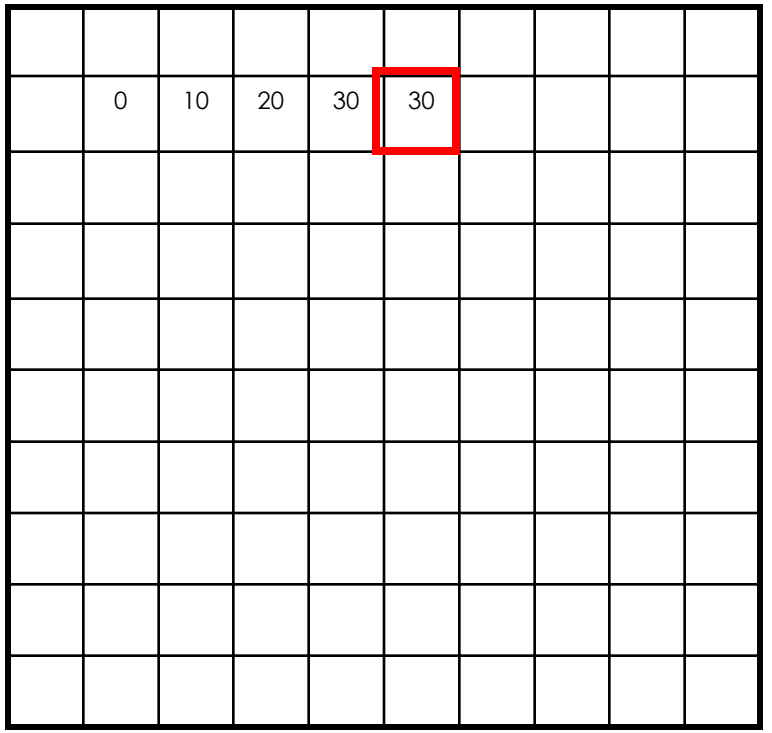
$K[\cdot\,,\cdot\,]$

$$\frac{1}{9}\begin{bmatrix}1&1&1\\1&1&1\\1&1&1\end{bmatrix}$$

$I[.,.]$



$S[.,.]$



$$S(i,j) = (I * K)(i,j) = \sum_{m=-p}^{p}\sum_{n=-p}^{p} I(i+m, j+n)K(m,n)$$

Credit: S. Seitz

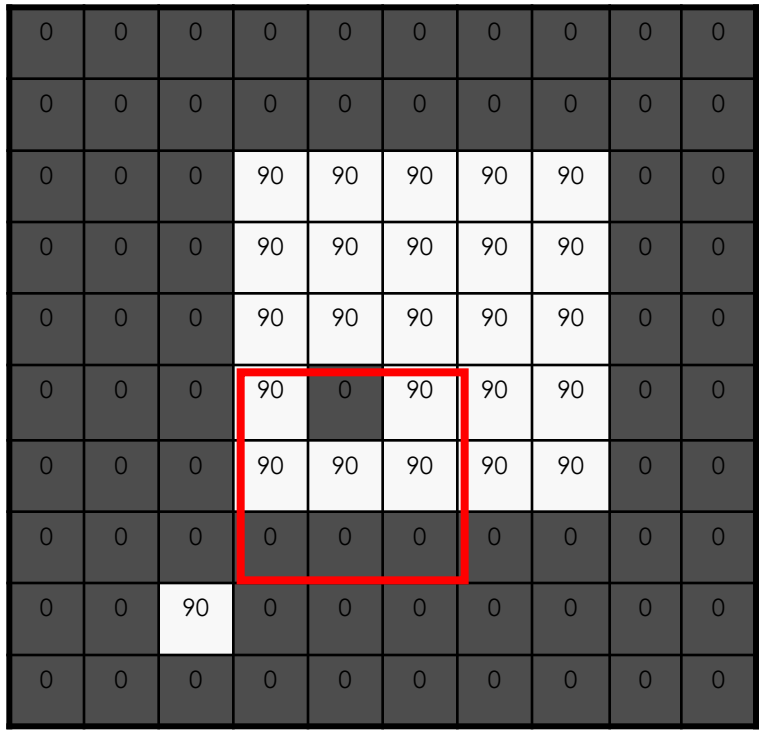
Image filtering

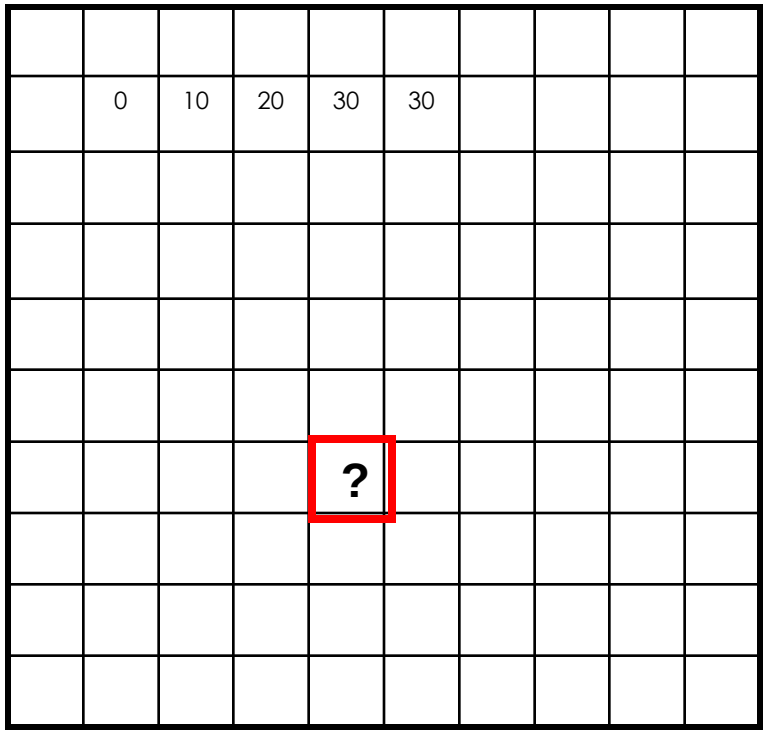
$g[\cdot\,,\cdot\,]$

$$\frac{1}{9}\begin{bmatrix}1&1&1\\1&1&1\\1&1&1\end{bmatrix}$$

$f[.,.]$

$h[.,.]$





$$S(i,j) = (I * K)(i,j) = \sum_{m=-p}^{p}\sum_{n=-p}^{p} I(i+m, j+n)K(m,n)$$

Credit: S. Seitz

Image filtering

$K[\cdot\,,\cdot\,]$ $\frac{1}{9}$

1	1	1
1	1	1
1	1	1

$I[.,.]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$S[.,.]$

	0	10	20	30	30				
						?			
				50					

$$S(i,j) = (I * K)(i,j) = \sum_{m=-p}^{p} \sum_{n=-p}^{p} I(i+m, j+n)K(m,n)$$

Credit: S. Seitz

Image filtering

$K[\cdot\,,\cdot\,]$ $\frac{1}{9}$

1	1	1
1	1	1
1	1	1

$I[.,.]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$S[.,.]$

	0	10	20	30	30	30	20	10	
	0	20	40	60	60	60	40	20	
	0	30	60	90	90	90	60	30	
	0	30	50	80	80	90	60	30	
	0	30	50	80	80	90	60	30	
	0	20	30	50	50	60	40	20	
	10	20	30	30	30	30	20	10	
	10	10	10	0	0	0	0	0	

$$S(i,j) = (I * K)(i,j) = \sum_{m=-p}^{p} \sum_{n=-p}^{p} I(i+m, j+n)K(m,n)$$

Credit: S. Seitz

Box Filter

What does it do?

- Replaces each pixel with an average of its neighborhood

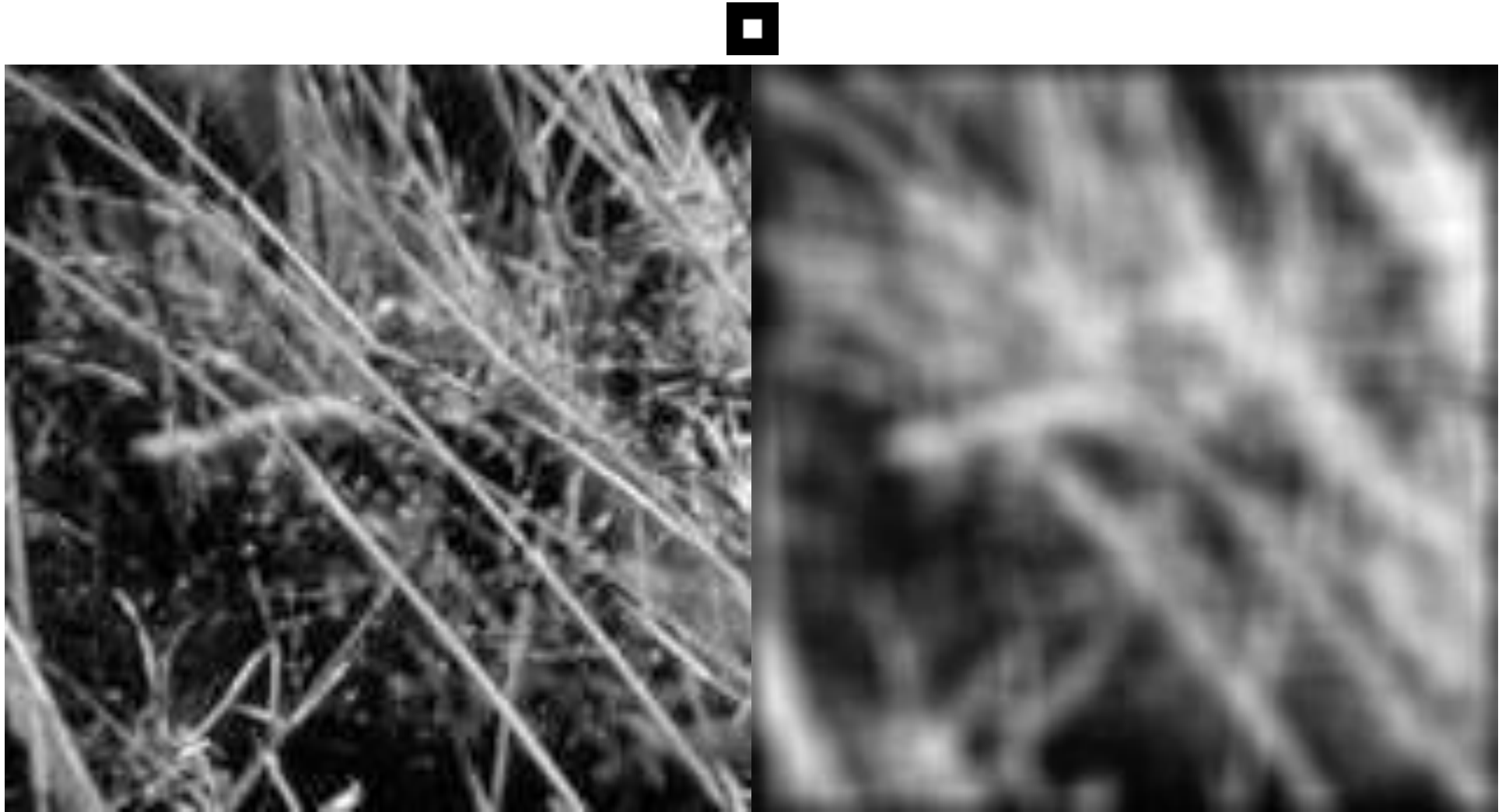
- Achieve smoothing effect (remove sharp features)

$K[\cdot\,,\cdot\,]$

$$\frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Slide credit: David Lowe (UBC)

Smoothing with box filter



Practice with linear filters

Original

0	0	0
0	1	0
0	0	0

?

Source: D. Lowe

Practice with linear filters

Original

0	0	0
0	1	0
0	0	0

Filtered
(no change)

Source: D. Lowe

Practice with linear filters

Original

0	0	0
0	0	1
0	0	0

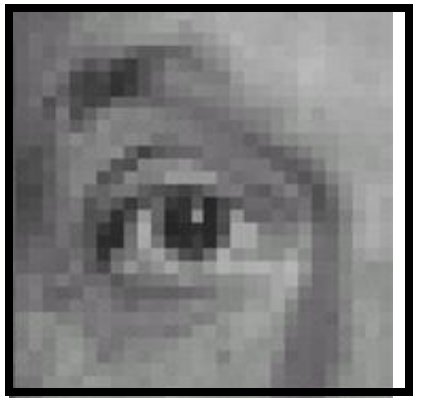
?

Source: D. Lowe

Practice with linear filters

Original

0	0	0
0	0	1
0	0	0



Shifted left
By 1 pixel

Source: D. Lowe

Practice with linear filters

Original

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 0 \end{bmatrix} - \frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} \quad ?$$

(Note that filter sums to 1)

Source: D. Lowe

Practice with linear filters

Original

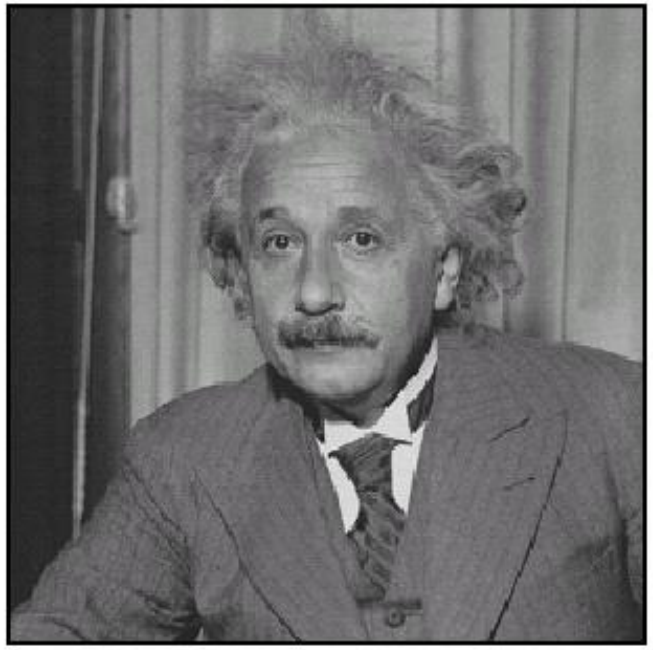
$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 0 \end{bmatrix} - \frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Sharpening filter

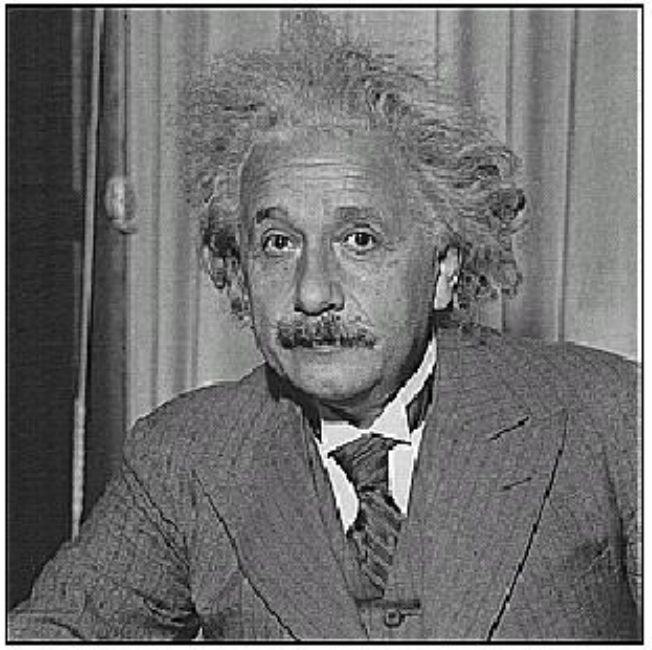
- Accentuates differences with local average

Source: D. Lowe

Sharpening



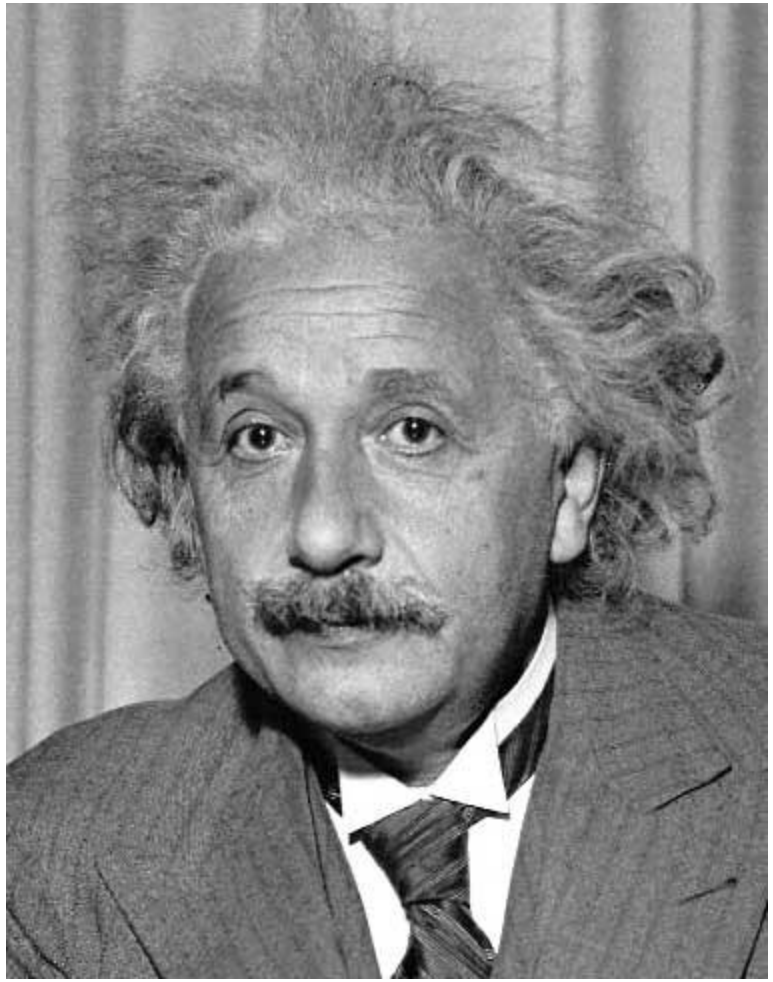
before



after

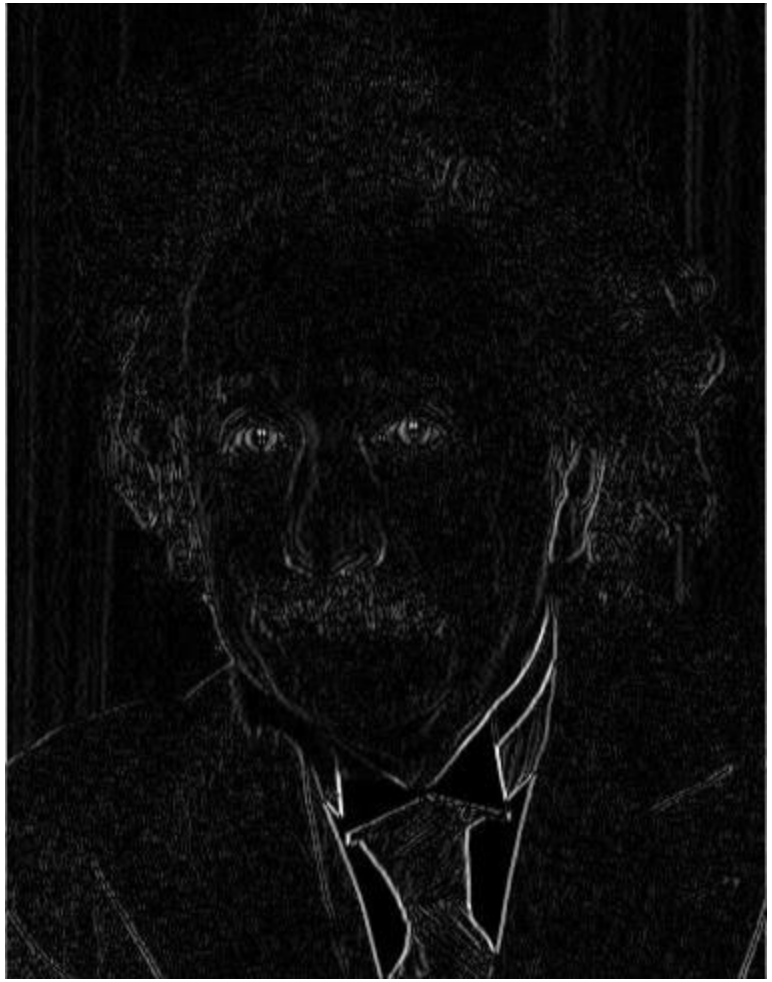
Source: D. Lowe

Other filters



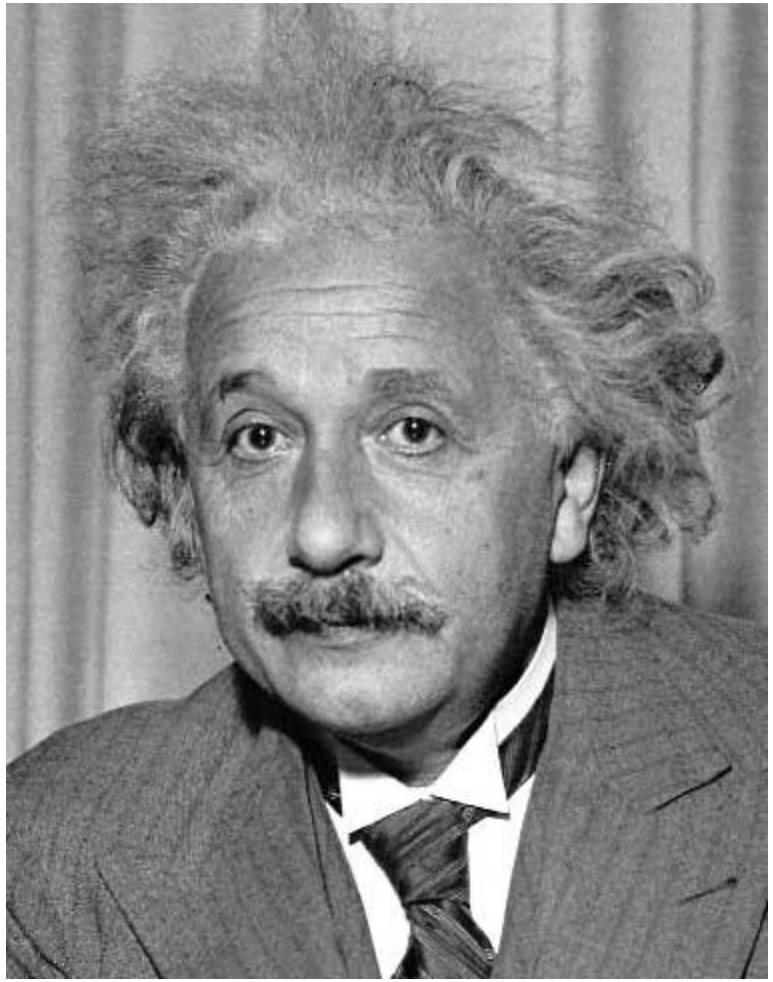
1	0	-1
2	0	-2
1	0	-1

Sobel



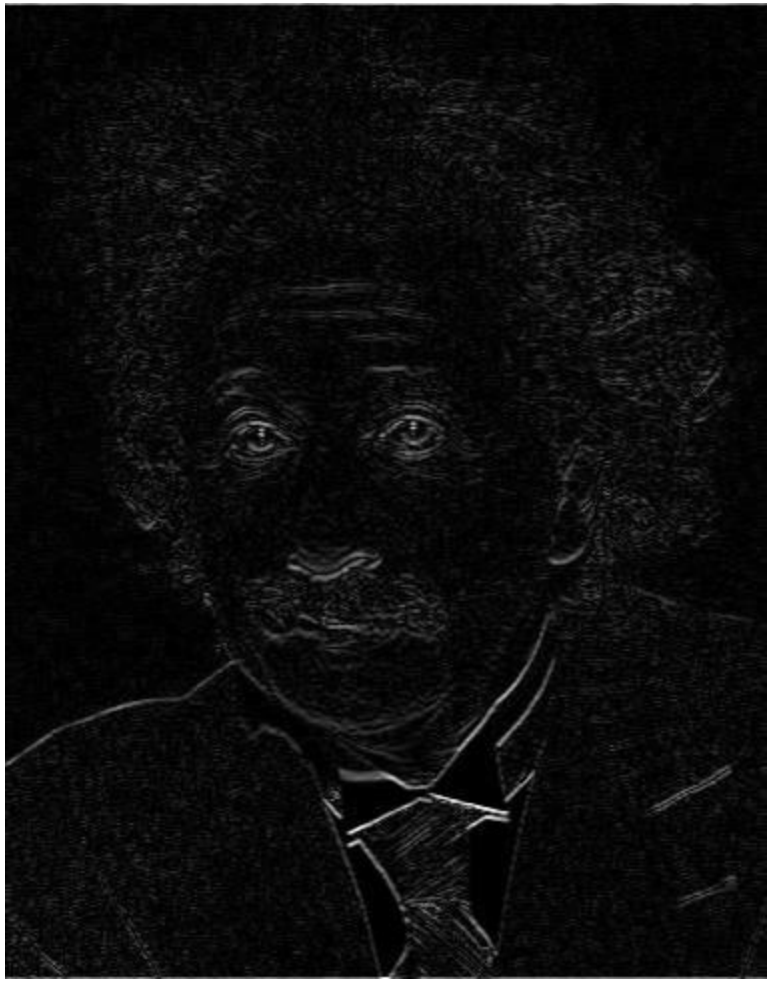
Vertical Edge
(absolute value)

Other filters



1	2	1
0	0	0
-1	-2	-1

Sobel



Horizontal Edge
(absolute value)

Filtering and Spectral Properties of Images

Fourier, Joseph (1768-1830)

French mathematician who discovered that any periodic motion can be written as a superposition of sinusoidal and cosinusoidal vibrations. He developed a mathematical theory of heat in *Théorie Analytique de la Chaleur (Analytic Theory of Heat),* (1822), discussing it in terms of differential equations.

Fourier was a friend and advisor of Napoleon. Fourier believed that his health would be improved by wrapping himself up in blankets, and in this state he tripped down the stairs in his house and killed himself. The paper of Galois which he had taken home to read shortly before his death was never recovered.

SEE ALSO: Galois

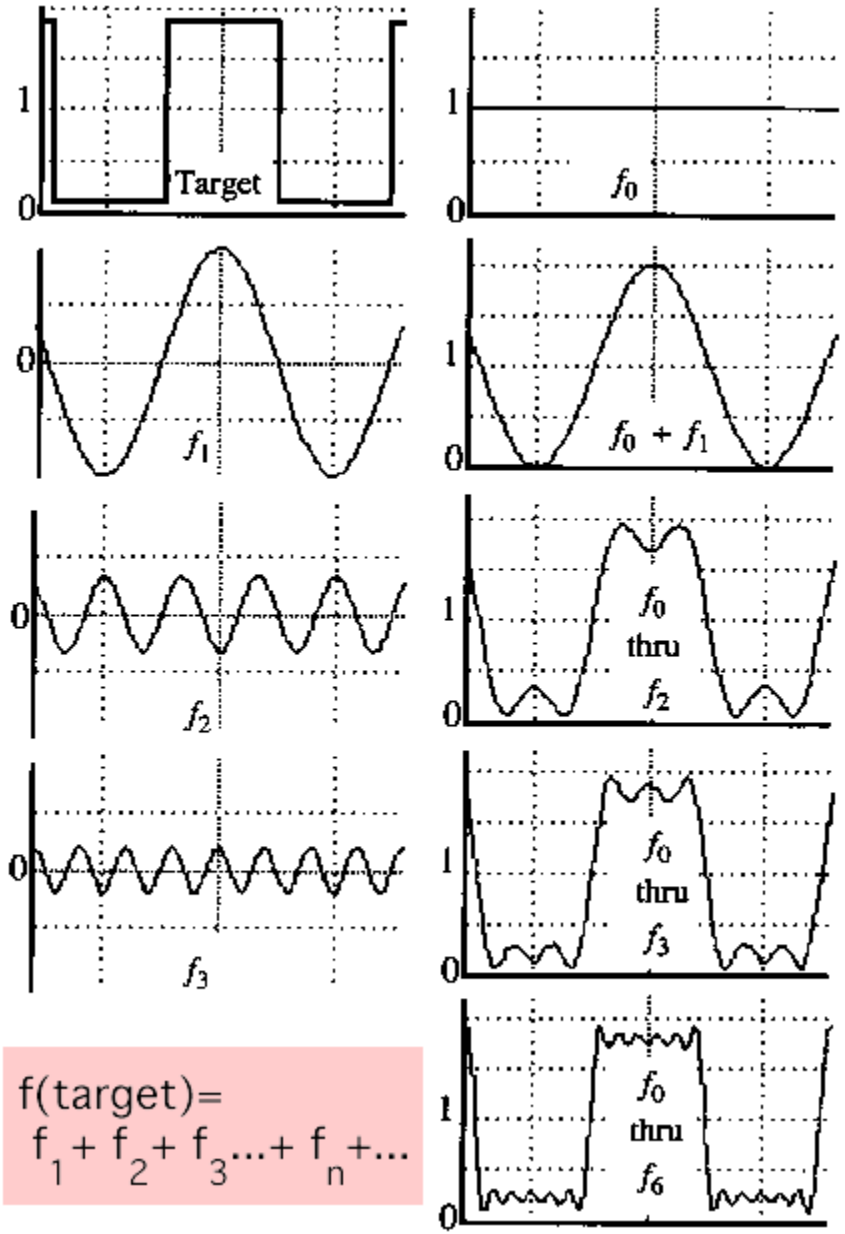
Additional biographies: MacTutor (St. Andrews), Bonn

A sum of sines

Our building block:

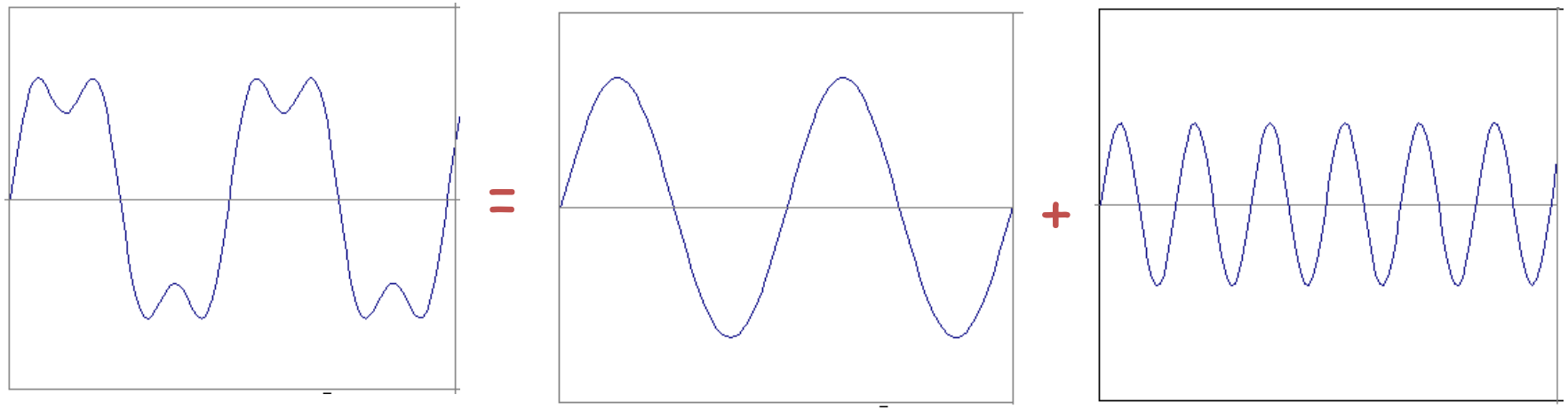
$$A\sin(\omega x + \varphi)$$

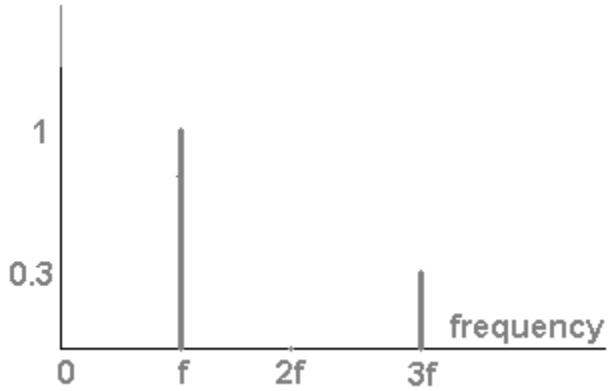
Add enough of them to get any signal *g(x)* you want!



Frequency Spectra

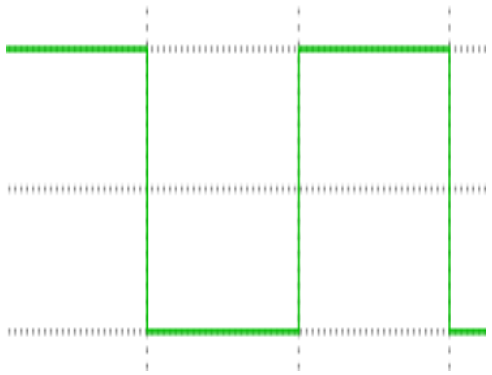
- example : $g(t) = \sin(2\pi f\, t) + (1/3)\sin(2\pi(3f)\, t)$



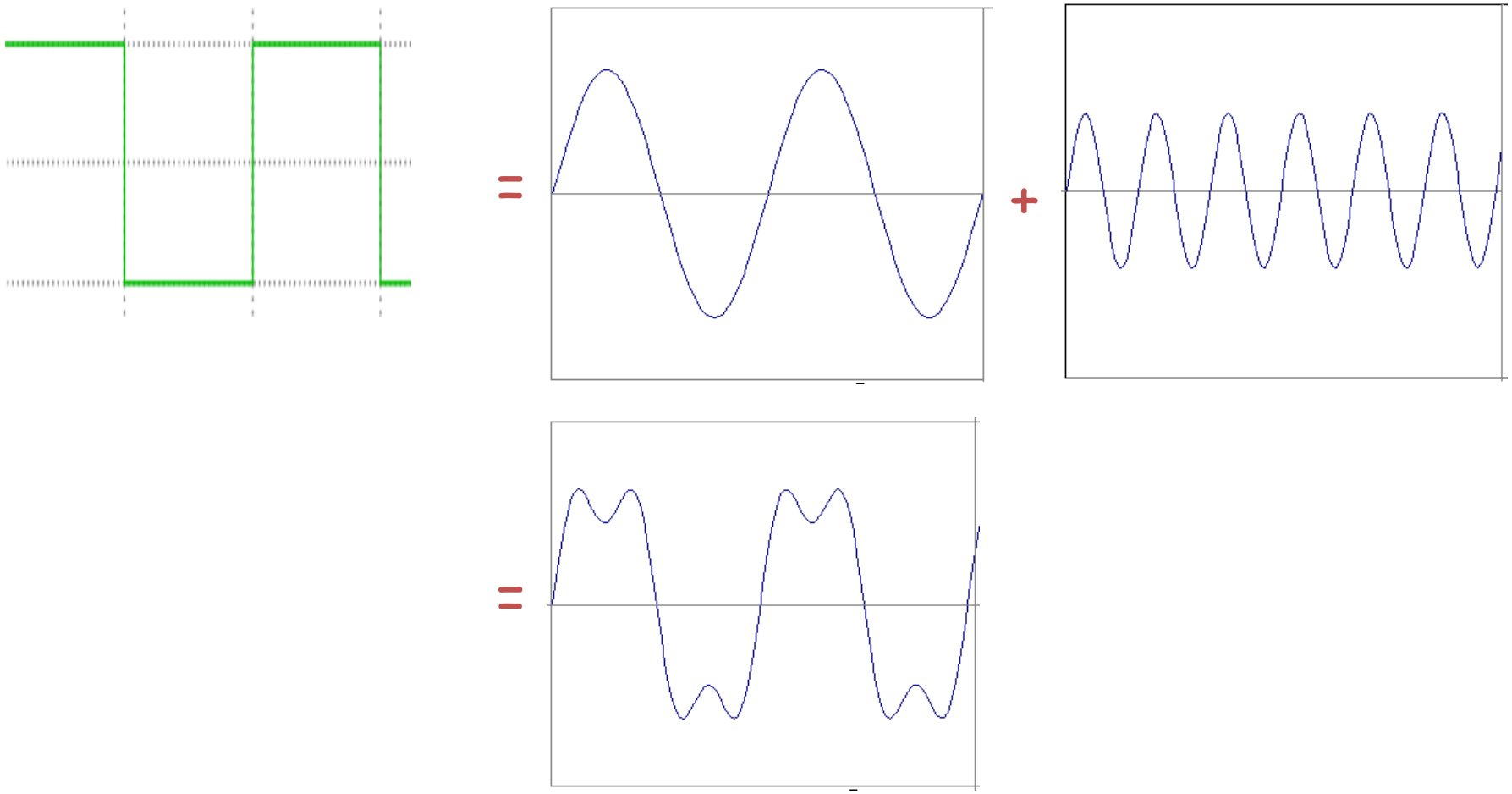


Slides: Efros

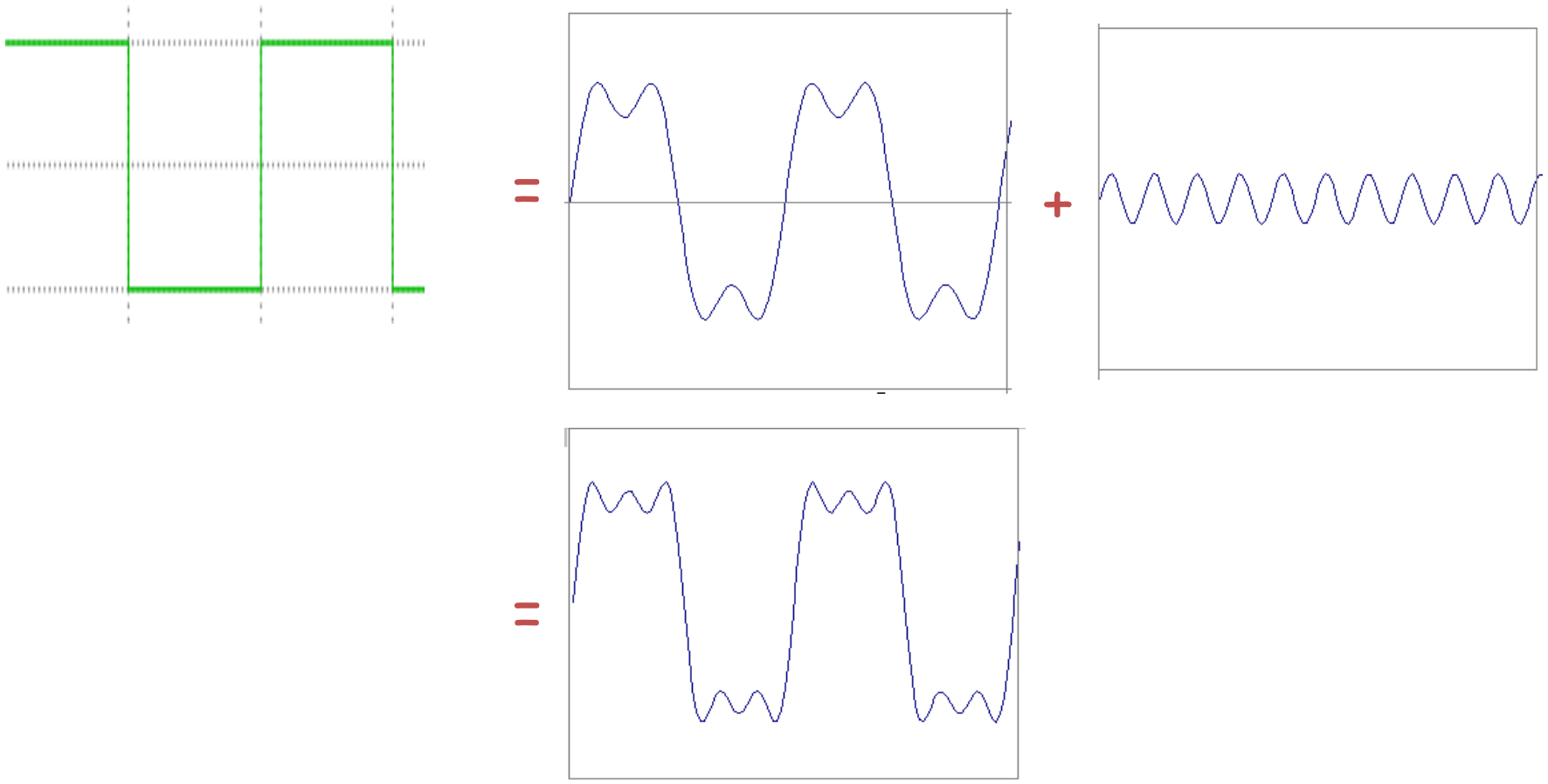
Frequency Spectra



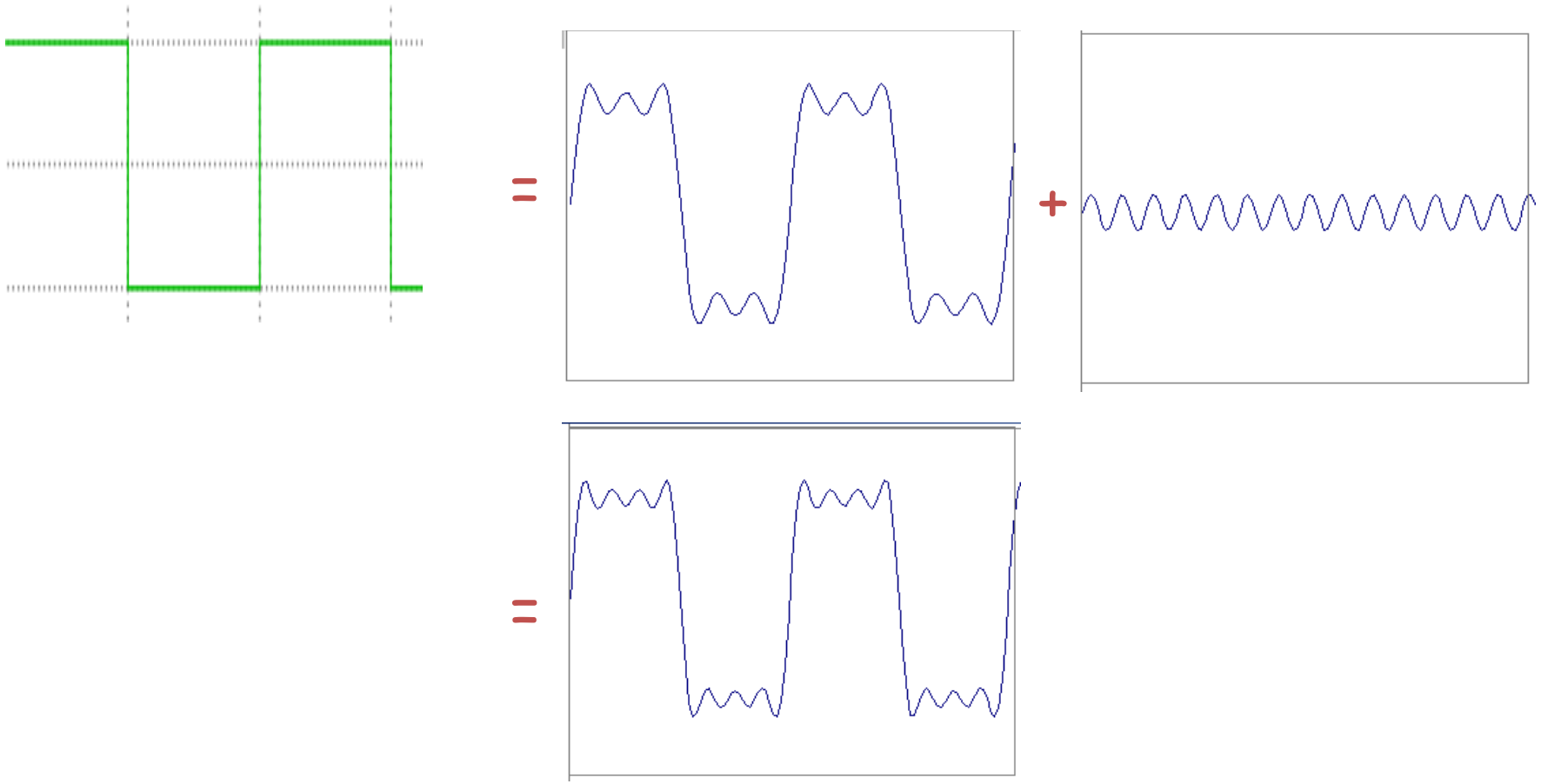
Frequency Spectra



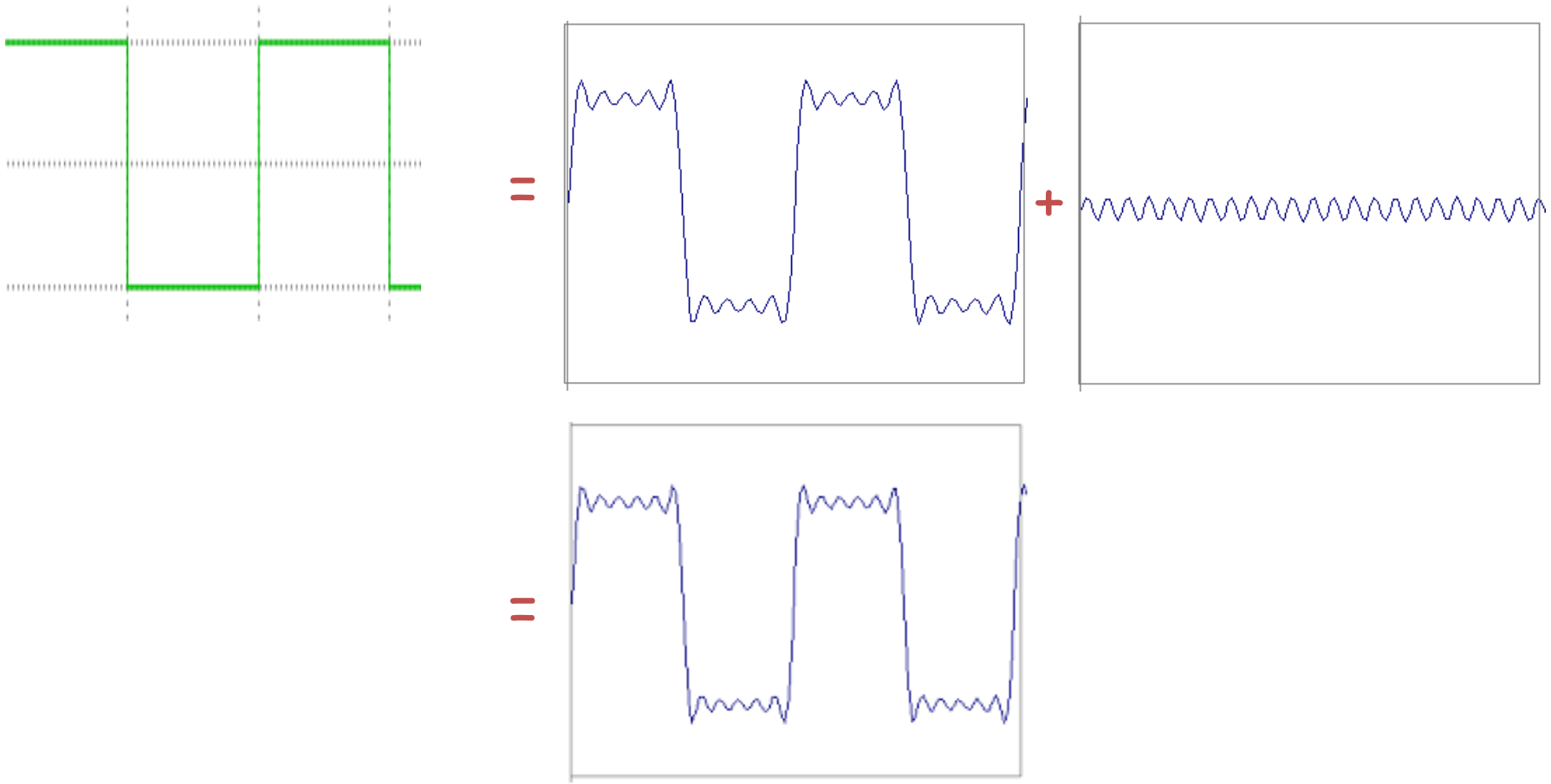
Frequency Spectra



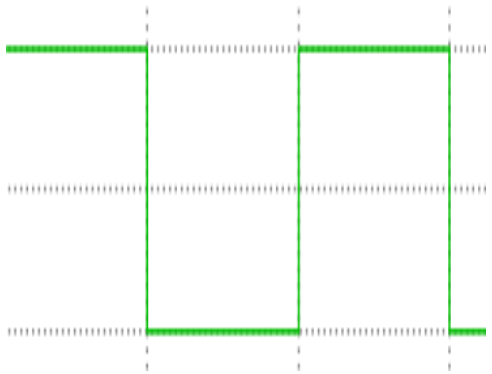
Frequency Spectra

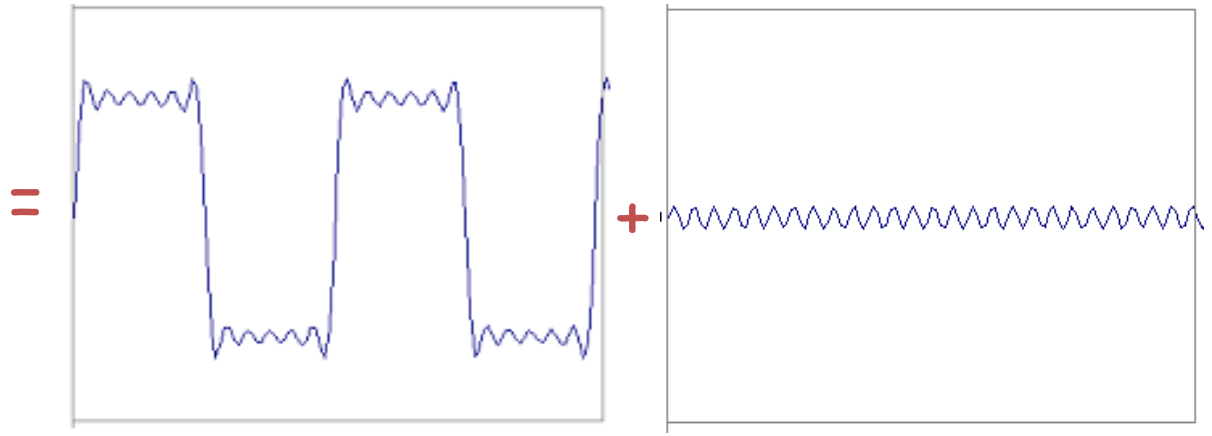


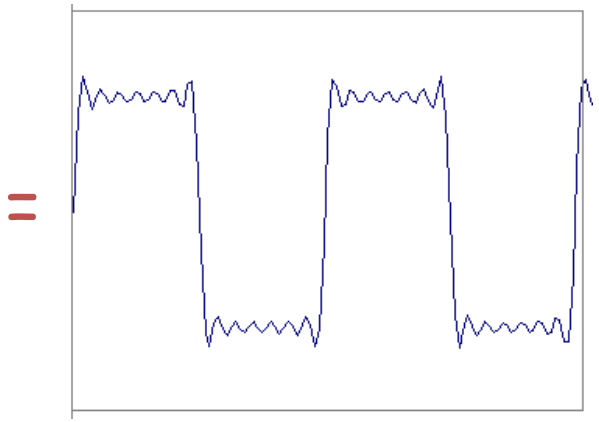
Frequency Spectra



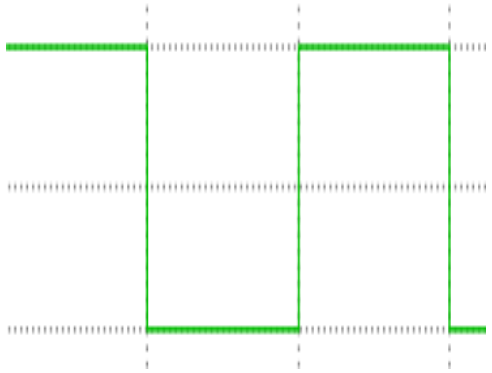
Frequency Spectra



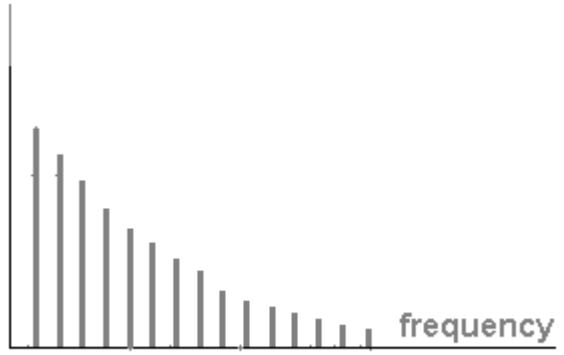




Frequency Spectra



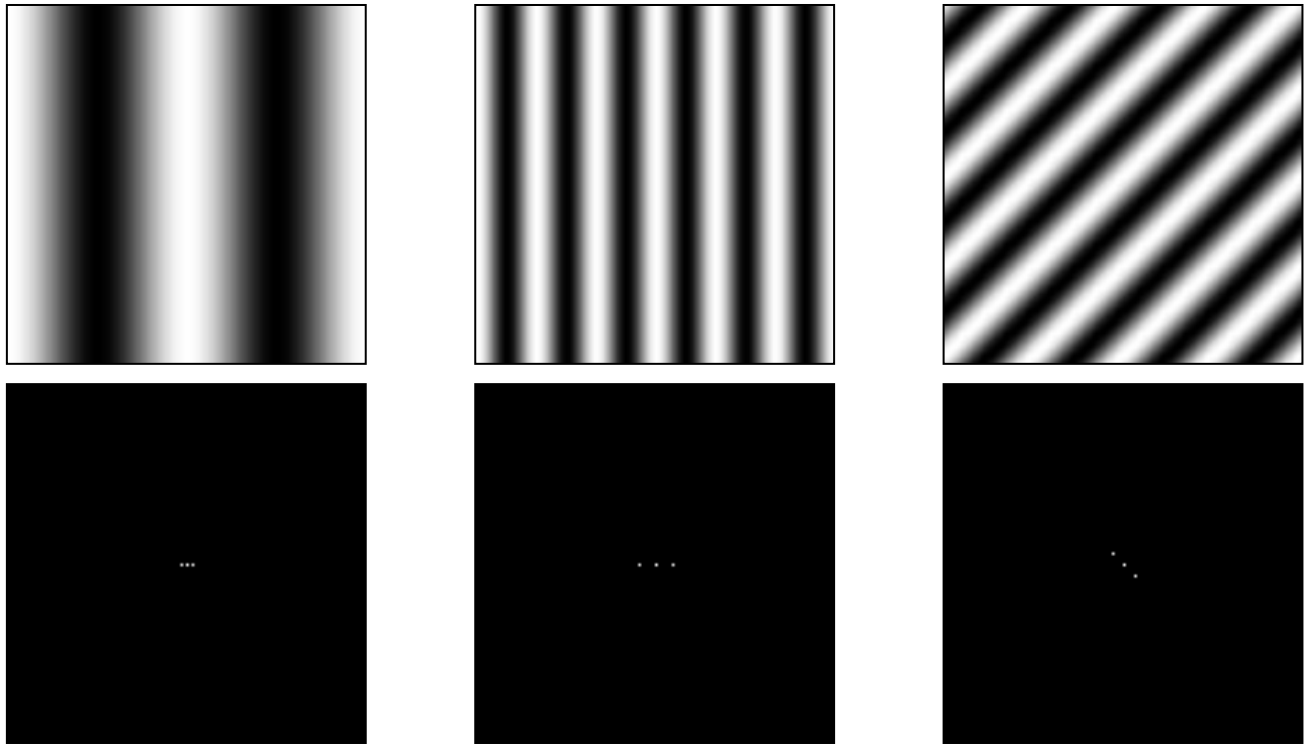
$$= A\sum_{k=1}^{\infty}\frac{1}{k}\sin(2\pi kt)$$



Fourier analysis in images

Intensity Image

Fourier Image



http://sharp.bu.edu/~slehar/fourier/fourier.html#filtering

Spectral Properties of Images

The **power spectrum** of an image reveals fundamental properties

- Power spectrum reveals the vertical and horizontal frequency components of the image
- **Low frequency** components of an image represent regions with slowly changing properties
- **High frequency** components of an image arise from **local features** with rapid changes
  - Edges
  - Corners
  - Texture
- **Filtering** changes the power spectrum of an image

Spectral Properties of Images

We can decompose an image into frequency components using a **Fourier transform**

- The Fourier transform decomposes a signal into frequency components
- The 1-dimensional Fourier transform of a function, $f(x)$, can be expressed:

$$\hat{f}(\omega) = \int_{-\infty}^{\infty} f(x)\, e^{-2\pi i x \omega} dx$$

- $\hat{f}(\omega)$ is the **complex spectrum** of $f(x)$
  - **Imaginary component** is **phase** at frequency $\boldsymbol{\omega}$
  - **Magnitude** of the complex number is amplitude at frequency $\boldsymbol{\omega}$

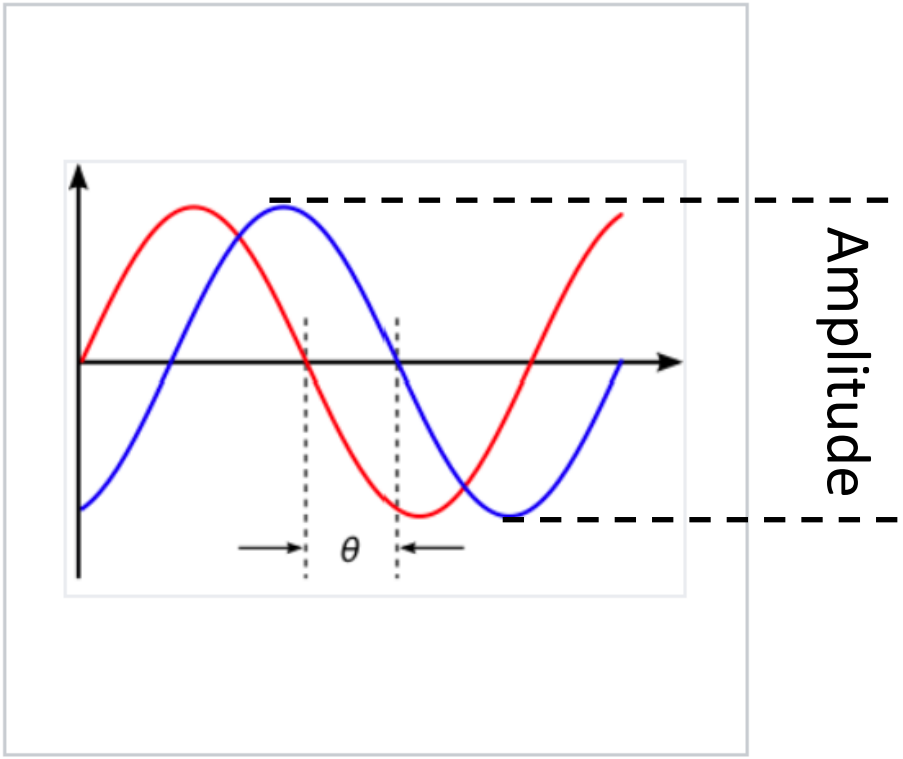
Spectral Properties of Images

We can decompose an image into frequency components using a **Fourier transform**

- The Fourier transform decomposes a signal into frequency components

$$\hat{f}(\omega) = \int_{-\infty}^{\infty} f(x)\, e^{-2\pi i x \omega} dx$$

- $\hat{f}(\omega)$ is the **complex spectrum** of $f(x)$
  - **Imaginary component** is **phase** at frequency ω
  - **Magnitude** of the complex number is the **signal amplitude** at frequency ω



Two sinusoidal signals with identical frequency and amplitude, phase shifted by θ

Credit, Wikipedia commons

Spectral Properties of Images

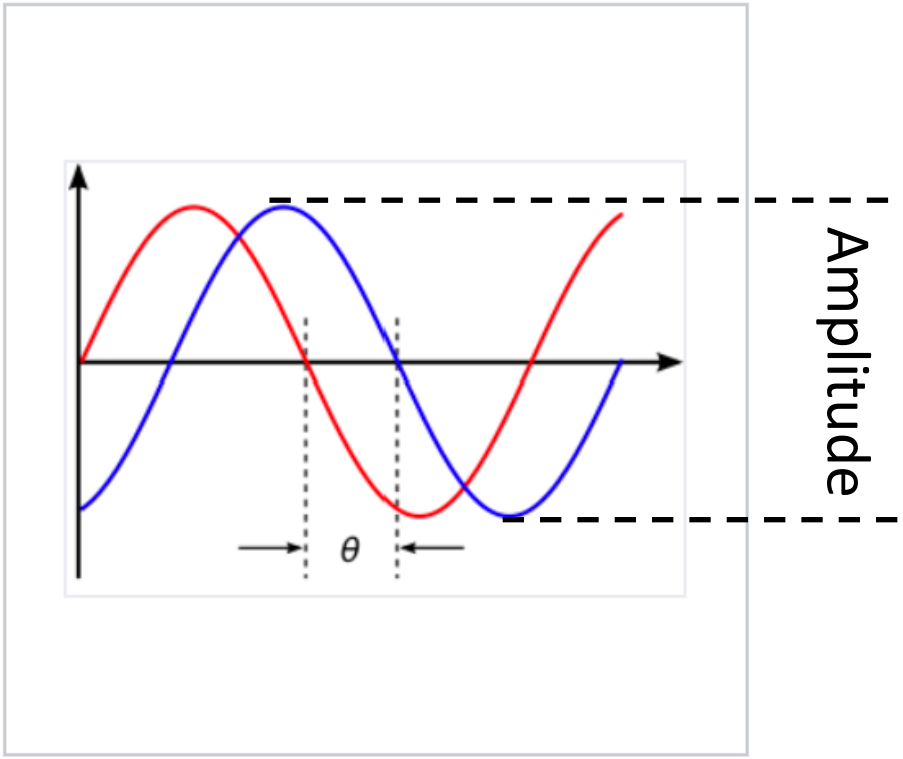
We can decompose an image into frequency components using a **Fourier transform**

- The Fourier transform decomposes a signal into frequency components

$$\hat{f}(\omega) = \int_{-\infty}^{\infty} f(x)\, e^{-2\pi i x \omega} dx$$

- For our purposes, we care about the **power spectrum**

$$\overline{S_\omega}(\omega) \triangleq \left|\hat{f}(\omega)\right|^2$$

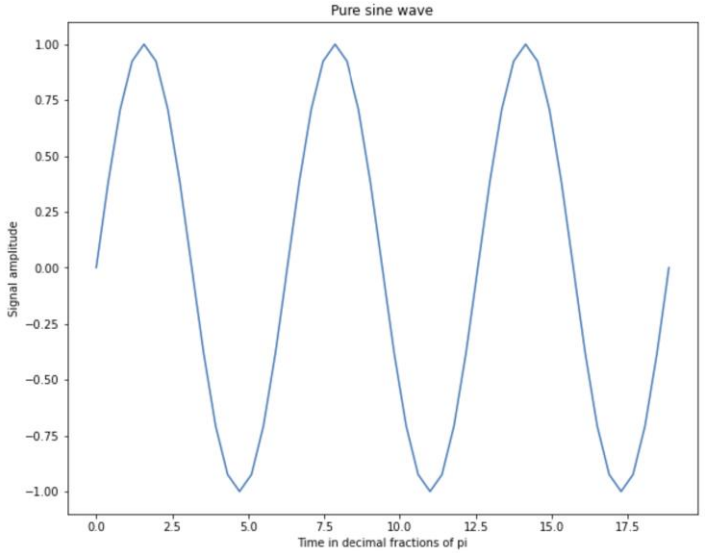


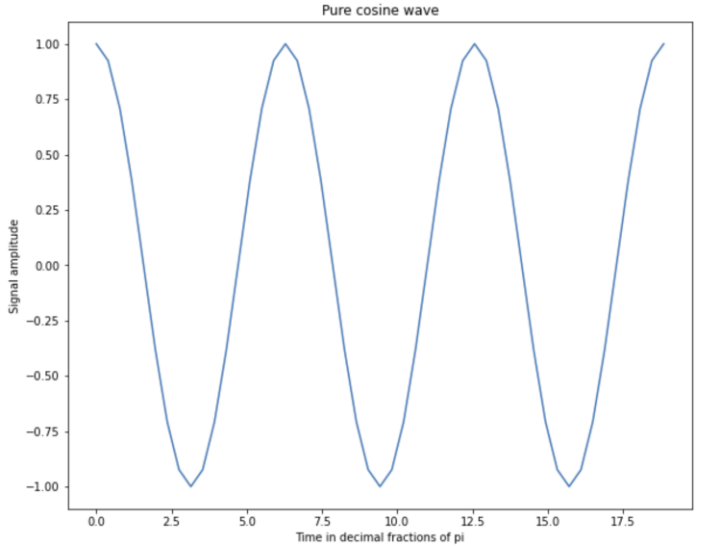
Two sinusoidal signals with identical frequency and amplitude, phase shifted by θ

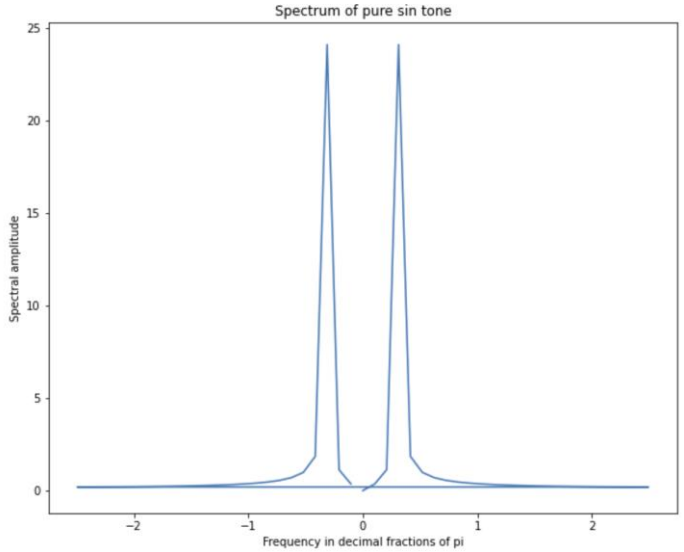
Credit, Wikipedia commons

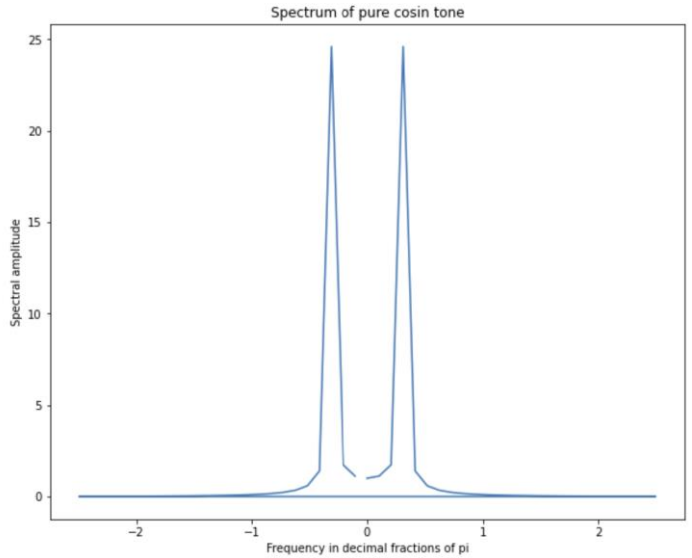
Spectral Properties of Images

Example of single frequency signals and their **power spectrum**



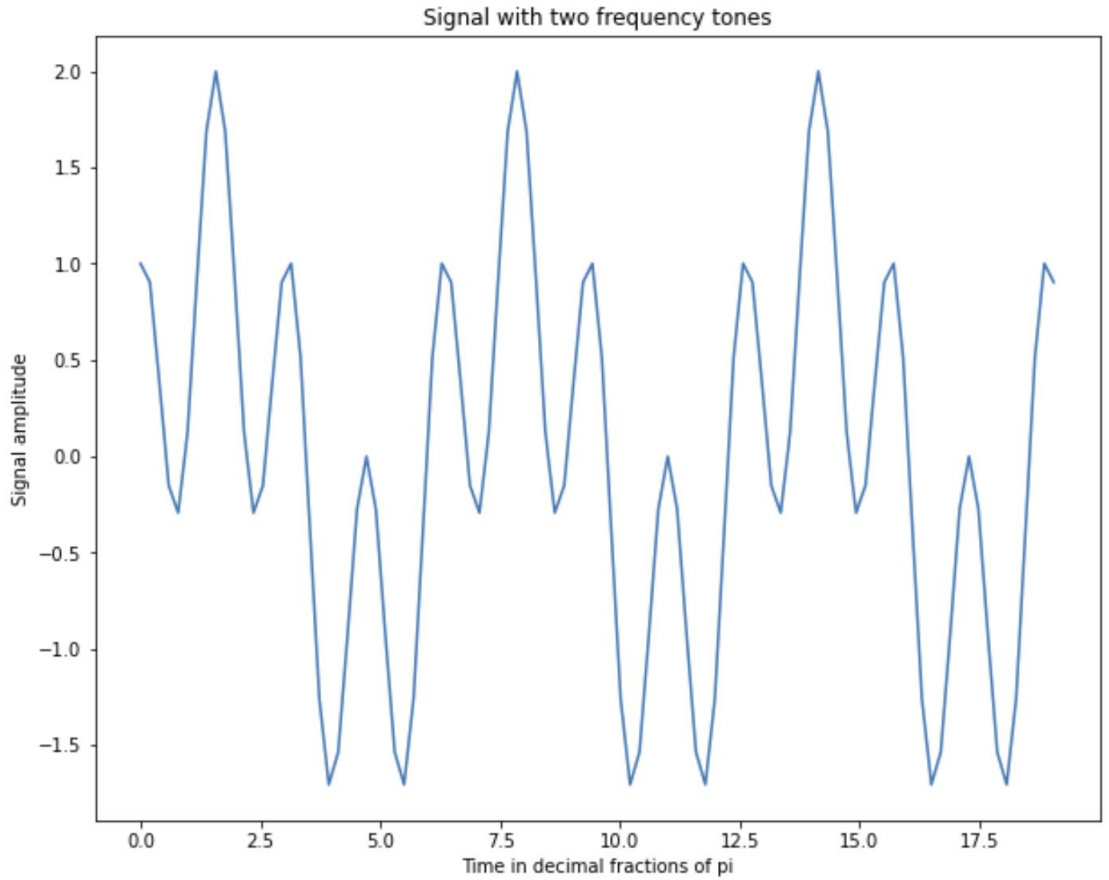


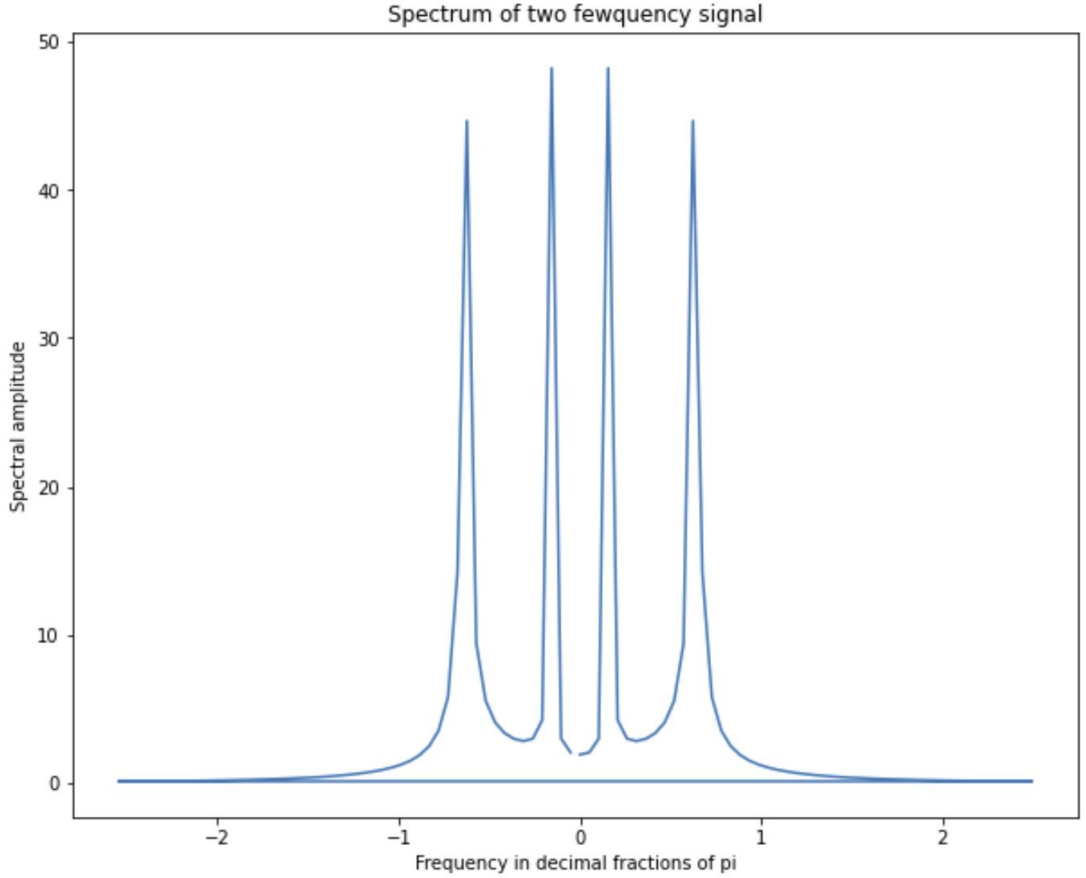




Spectral Properties of Images

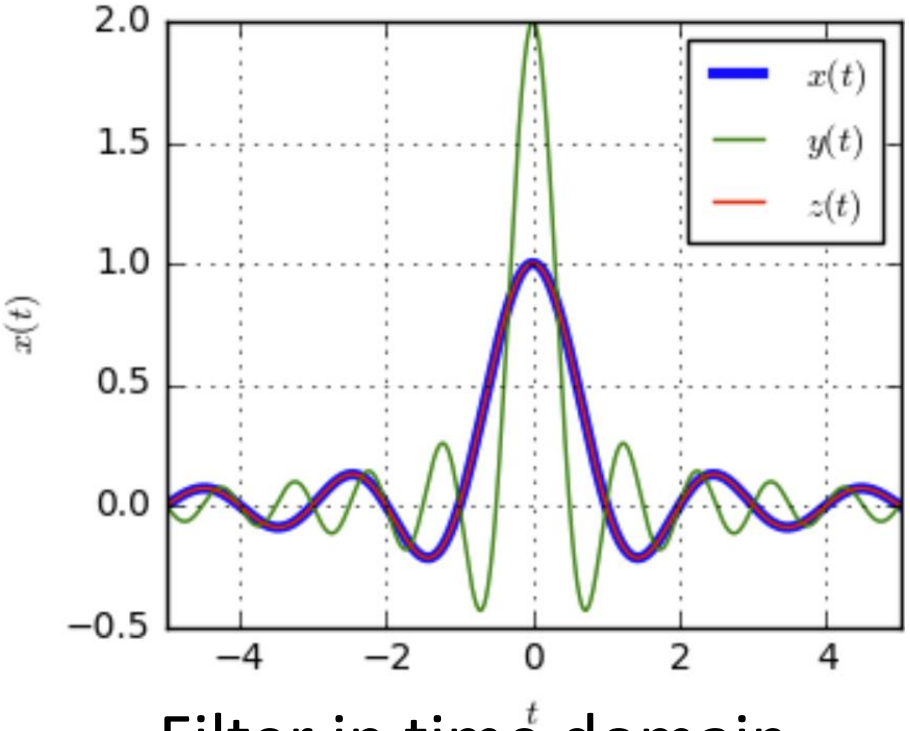
Example of signal with two frequencies and the **power spectrum**





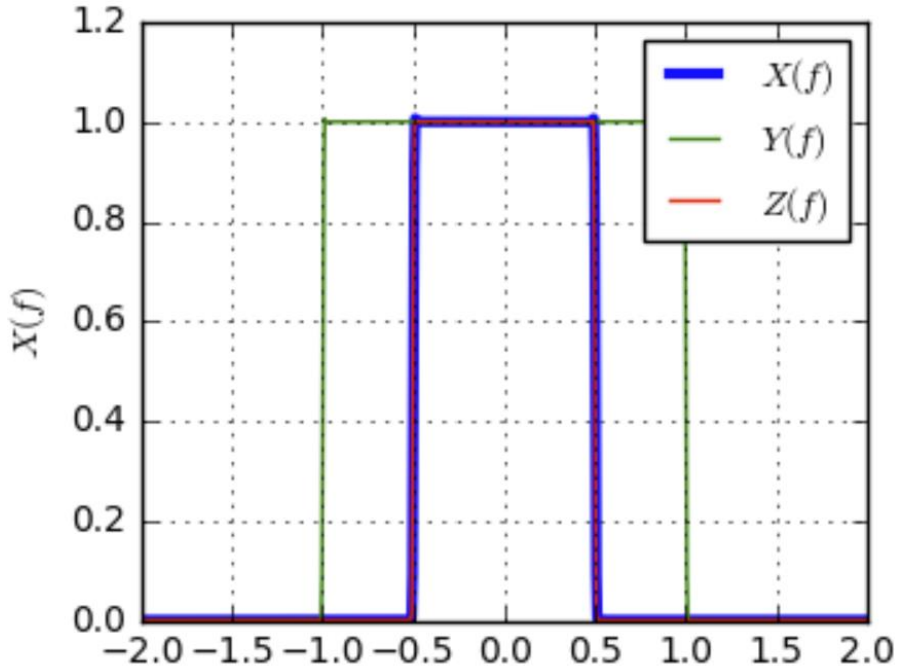
Spectral Properties of Images

Impulse response of convolutional a filter is the Fourier transform in the **frequency domain**



Filter in time domain

Fourier Transform



Filter in frequency domain

Credit, DSP Illustrations

Relationship is reversable (inverse Fourier transform) and commutable – exchange time and frequency domain

Spectral Properties of Images

For discretely sampled images we use a **2-dimensional discrete Fourier transform**

- The discrete Fourier transform is sampled over image samples (pixels), n_x, n_y, and discrete frequencies, K_x, K_y, within the Nyquist limit:

$$\hat{f}(K_x, K_y) = \sum_{n_x=0}^{N_x-1} \sum_{n_y=0}^{N_y-1} f(n_x, n_y)\, e^{-i\frac{2\pi}{N_x} n_x K_x - i\frac{2\pi}{N_y} n_y K_y}$$

  - (n_x, n_y) is the spatial index of a discrete pixel
  - K_x, K_y are discrete horizontal and vertical frequencies
- Notice the limited spatial range of the sampling
  - Not $\{-\infty, +\infty\}$
  - Limited spatial sampling introduces edge artifacts – edges of images are impulses

Spectral Properties of Images

For a discretely sampled image we use a **2-dimensional discrete Fourier transform**

- The discrete Fourier transform is sampled over image samples (pixels), n_x, n_y, and discrete frequencies, K_x, K_y, within the Nyquist limit:

$$\hat{f}(K_x, K_y) = \sum_{n_x=0}^{N_x-1} \sum_{n_y=0}^{N_y-1} f(n_x, n_y)\, e^{-i\frac{2\pi}{N_x} n_x K_x - i\frac{2\pi}{N_y} n_y K_y}$$

- Appling Euler's formula, $e^{-x} = \cos(x) - i\sin(x)$, we get:

$$\hat{f}(K_x, K_y) = \sum_{n_x=0}^{N_x-1} \sum_{n_y=0}^{N_y-1} f(n_x, n_y) \left[\cos\left(\frac{2\pi}{N_x} n_x K_x - \frac{2\pi}{N_y} n_y K_y\right) - i\sin\left(\frac{2\pi}{N_x} n_x K_x - \frac{2\pi}{N_y} n_y K_y\right)\right]$$

- This form makes the real (cos) and imaginary or phase (sin) components explicit

Spectral Properties of Images

The **power spectrum** of an image reveals fundamental properties

- **Spectral density** reveals the **power or energy** in vertical and horizontal frequency components of the image

$$\overline{S_{K_x,\,K_y}}(K_x,\,K_y) \triangleq \left|\hat{f}(K_x,\,K_y)\right|^2$$

- In words, the spectral density is the lag-1 autocorrelation of the estimated Fourier transform, $\hat{f}(K_x,\,K_y)$
- Or square of the absolute value of the estimated Fourier transform

Spectral Properties of Images

The **power spectrum** of an image reveals fundamental properties

- Spectral density reveals the **power or energy** in vertical and horizontal frequency components of the image given the estimated Fourier transform, $\hat{f}(\omega)$

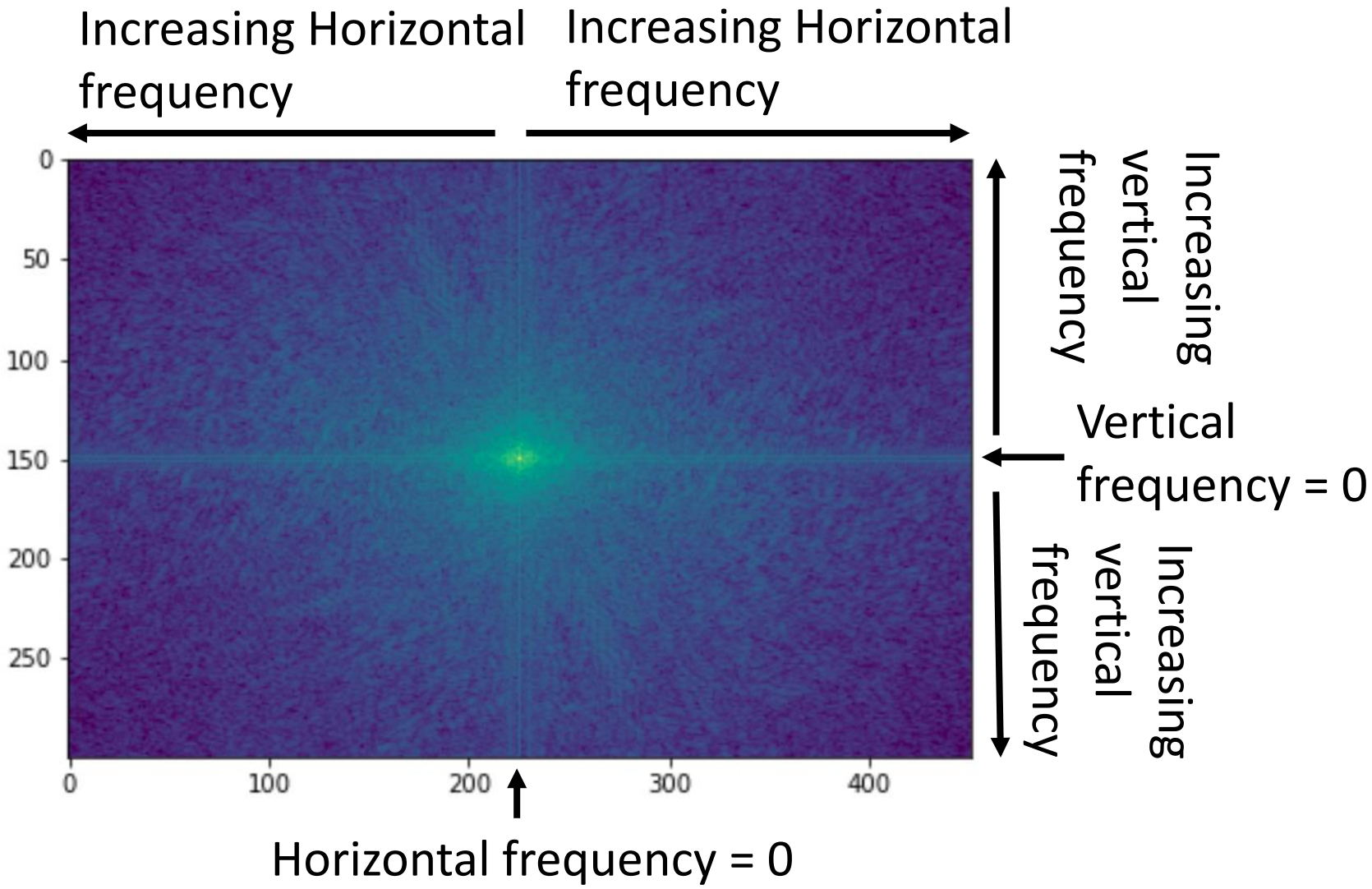
$$\overline{S_{K_x,\,K_y}}(K_x,\,K_y) \triangleq \left|\hat{f}(K_x,\,K_y)\right|^2$$

- Spectral density is easier to work with, since it just gives us the power at each frequency, $K_x,\ K_y$
  - The squared magnitude of the signal at $K_x,\ K_y$
- Notice that, unlike Fourier transform, the spectral density is not invertible
  - The square of absolute magnitude of $\hat{f}(K_x,\,K_y)$ loses phase information

Spectral Properties of Images

Spectral density reveals the **power or energy** in vertical and horizontal frequency components of the image

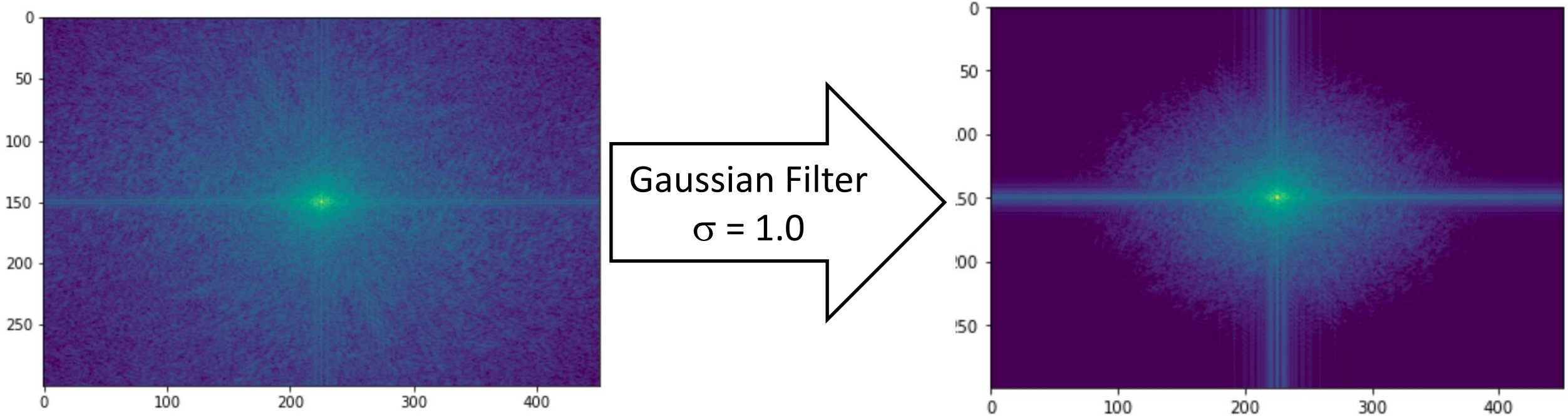
- For real-valued image, spectral density is symmetric about zero frequency
- Rapid changes (e.g. edges, corners) have high frequency components



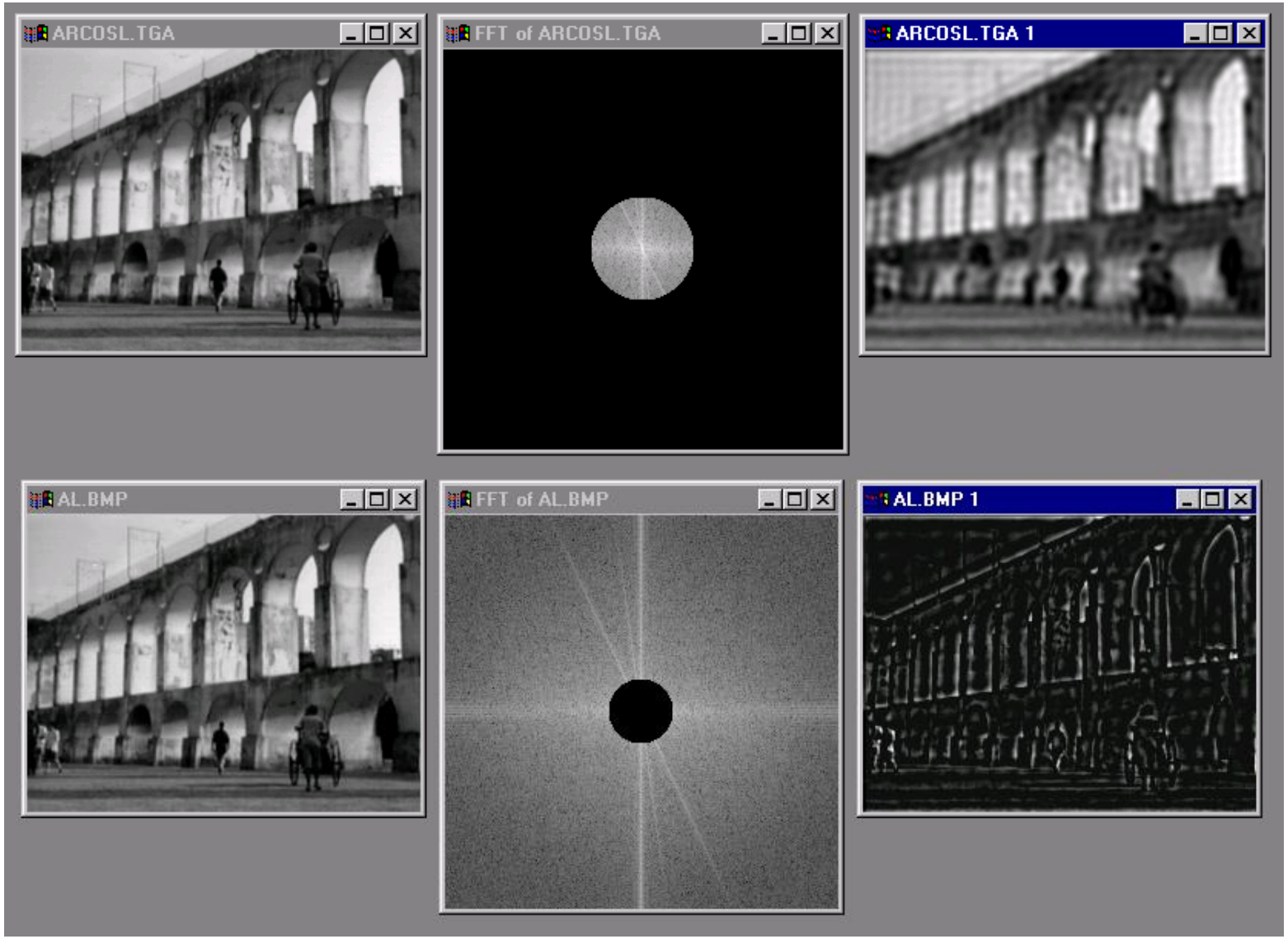
Spectral Properties of Images

Changes in spectral density show the **effect of filtering**

- Filters change the spectral characteristics of images
- Filters can **suppress unwanted high frequency noise**
- Filtering **reduces aliasing**
- Lower frequency image is **blurred** – edges and corners less sharp



Low and High Pass filtering



The Convolution Theorem

There is a direct relationship between filtering in the spatial (or time) domain and the frequency domain, the **convolution theorem**

- For spatial domain image, $f(x, y)$, its Fourier transform, $\hat{f}(\omega_x, \omega_y)$, operating on by a spatial domain convolutional kernel, $k(x, y)$, its Fourier transform, $\hat{f}(\omega_x, \omega_y)$, we can state the convolution theorem:

$$(f * k)(x, y) \Leftrightarrow \hat{f}^{-1}(\hat{f} \cdot \hat{k})$$
$$(f \cdot k)(x, y) \Leftrightarrow \hat{f}^{-1}(\hat{f} * \hat{k})$$

- In words, the convolution theorem states that convolution in the spatial domain is equivalent to the inverse Fourier transform of the point wise product in the frequency domain, or vice versa
- The convolution theorem helps us understand the effects of filtering

Summary

Key points for this lesson

- How convolutional works in 1-d, 2-d and higher dimension
- How to apply padding, stride and tiling for convolution
- Constructing convolutional filter kernels
- The relationship between filtering and the spectrum of images